
Entity-Centric Scientific Retrieval over a Temporal Knowledge Hypergraph

A High-Recall Multi-Channel Retriever with
Cross-Encoder Reranking

SUPPLEMENTARY TECHNICAL REPORT

Technical Supplement to *Temporal Semantic Hypergraph Coarsening*

Luka Burduli

October 2026

Supplementary report for the downstream retrieval component, package `clean_tshc_v2`.

It explains the current V2 retrieval architecture (configuration “A6”), how every component works, how predictions are isolated from ground truth, and what the frozen development results do and do not show. It does *not* replace the main TSHC report: the hierarchy, its guarantees and its intrinsic evaluation are established there and are unchanged.

Abstract

The main report builds a laminar, budgeted, temporally regularised hierarchy over a temporal knowledge hypergraph of 5,798 nodes and 1,429 hyperedges. Its extrinsic evaluation exposed a separate problem: dense retrieval, flat or routed through the hierarchy, found the right scientific *region* but did not rank the exact answer *entity*; no expected method was recovered in the top 25. This supplement describes the retrieval architecture built in response. It treats two tasks separately. *Candidate admission* forms a high-recall pool of canonical answer entities from six inexpensive ranked lists (two dense views, BM25, safe name matching, evidence-to-entity retrieval and bounded graph expansion), combined by reciprocal rank fusion. *Candidate ordering* scores only the first 100 fused candidates with a pretrained relevance cross-encoder that reads the question together with an identity-first, query-dependent evidence card of at most 512 tokens.

No model is trained and no weight or threshold is fitted; the one choice made with development results in view is the reranking depth $K = 100$. Predictions are produced in a process that cannot read answers, are hashed, and are frozen before a separate evaluator loads the ground truth. On the development benchmark (14 scored questions, 63 target occurrences, 49 of them directly mapped to graph identities) the candidate pool admits 37 of 49 mapped targets, and the final ranking recovers 12 of 63 targets in the top 10, 18 in the top 25 and 37 in the top 100, with a mean reciprocal rank of 0.167. These are development diagnostics on a benchmark that influenced design; they are not held-out performance, and exact-answer ranking remains difficult.

Scope of this document

This document describes the current V2 retrieval architecture and its frozen development results. It is intentionally scoped to that system rather than to the broader exploratory history. Everything about the hierarchy itself (objective, laminarity, budgets, coherence, hyperedge collapse, temporal identities, labels) is inherited from the main report (Burduli, 2026) and is only summarised here.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Why V2 was needed	4
1.3	V2 is a supplement, not a replacement	5
1.4	Contributions of V2	6
1.5	How to read this document	6
2	Relation to the Assessment Requirements	6
2.1	Inherited guarantees: P1–P6	6
2.2	Tasks T1–T7	7
2.3	Scoring criteria	7
3	Retrieval Problem Formulation	7
3.1	Objects	7
3.2	Two stages, two metrics	8
4	Canonical Entity Representation	10
4.1	Why raw nodes are the wrong unit	10
4.2	Construction	10
4.3	Evidence links	10
5	Answer-Family Filtering	11
6	Two Query Views	13
7	First-Stage Candidate Generation	13
7.1	Dense semantic retrieval	13
7.2	BM25 lexical retrieval	14
7.3	Evidence-level dense retrieval	15
7.4	Safe name and alias channel	15
7.5	Bounded graph expansion	16
7.6	Summary and measured coverage	17
8	Multi-Channel Union and Rank Fusion	17
8.1	Reciprocal rank fusion	17
9	Candidate Admission versus Ranking	20
10	Cross-Encoder Reranking: the A6 Stage	20
10.1	Bi-encoder and cross-encoder	21
10.2	The model actually used	21
10.3	The ranking rule	22
11	The Query-Dependent Candidate Card	22
11.1	What the card contains	22
11.2	Packing into 512 tokens	23
12	The Complete A6 Algorithm	24
13	Worked End-to-End Example	25

14 Temporal and Hypergraph Integrity	27
14.1 Visibility	27
14.2 Provenance	28
14.3 What V2 does and does not do with hyperedges	28
15 Leakage Prevention	28
15.1 What the prediction phase may and may not see	28
15.2 Frozen models and no fitting	30
15.3 Freeze, then score	30
15.4 What the firewall does not establish	31
16 Evaluation Protocol	31
16.1 Targets and their status	31
16.2 Metrics	32
16.3 Is the evaluator itself sound?	32
17 Results	32
17.1 Headline numbers	33
17.2 Where targets are lost	33
17.3 What the reranking stage changed	34
17.4 Per-question view	34
18 Interpreting the Results	36
18.1 What A6 demonstrates	36
18.2 What A6 does not demonstrate	36
18.3 Verified versus assumed	36
19 Efficiency	37
20 Reproducibility	38
21 Limitations	40
22 What Four More Weeks Would Add	41
23 Conclusion	41
AI assistance	41
References	41
A Configuration Constants	43
B Artifacts and Sources Inspected	43
C Glossary	44

List of Figures

1	Why single-vector retrieval misses the answer identity	4
2	Relation between the main TSHC system and V2	5
3	Canonical entity construction	11
4	The two query views	13
5	Bi-encoder architecture of the dense channels	14
6	Evidence-to-entity retrieval	16
7	Permitted graph expansion paths	16
8	Multi-channel first-stage architecture	19
9	Toy reciprocal rank fusion example	19
10	Two-stage funnel: admission and ordering	20
11	Bi-encoder versus cross-encoder	21
12	Joint attention between question and candidate phrases	21
13	A query-dependent candidate card and its token packing	24
14	One candidate traced through A6	26
15	Prediction/evaluation firewall and SHA-256 freeze	29
16	Recorded order of freeze and evaluation events	30
17	Final recall at six cut-offs (development diagnostics)	33
18	Target loss by stage (development diagnostics)	34
19	Rank distribution of mapped targets (development diagnostics)	35
20	Cheap first stage, expensive second stage	38

List of Tables

1	Properties P1–P6 and their relation to V2	7
2	Tasks T1–T7 and their relation to V2	8
3	Scoring criteria and where they are addressed	9
4	Notation	9
5	The frozen 2026 retrieval index	12
6	Answer heads and admitted node types	12
7	The six first-stage lists	18
8	Measured coverage of mapped targets per list	18
9	Evidence families and caps of the candidate card	23
10	Top entries of each first-stage list for Q12	26
11	Final top ten for Q12	27
12	Target-status categories	31
13	A6 results (development diagnostics)	33
14	Effect of the reranking step (development diagnostics)	34
15	Per-question results (development diagnostics)	35
16	Verified versus assumed	37
17	Logical work and timing	37
18	Pinned models and environment	38
19	Tests guarding the properties relied upon	39
20	Configuration constants	43
21	Sources inspected	44

List of Algorithms

1	Bounded graph expansion	17
2	Card construction and token packing for one candidate	23
3	A6 retrieval	25

1 Introduction

1.1 Motivation

The assessment supplies a Temporal Knowledge Hypergraph (TKH) extracted from 52 papers on machine-learning interatomic potentials and computational materials science. It has 5,798 nodes of twelve types (methods, techniques, components, cited works, claims, tasks, problems, datasets, metrics, articles, authors, future topics) and 1,429 hyperedges of eleven relation types, with annual time stamps. The main report (Burduli, 2026) turns this graph into a three-level hierarchy of super-nodes at four snapshots and evaluates that hierarchy intrinsically.

The assessment also asks whether the abstraction helps a *downstream* task: given a scientific question such as “which methods are best suited for ...”, find the expected methods. The benchmark has 18 questions; 14 of them (Type A) come with a list of expected methods. This is the extrinsic part of task T6.

The main report ran that experiment honestly and reported a negative result. Under the declared rule that the expected method’s own node must appear in the top k , neither the flat dense baseline nor any hierarchical beam recovered a single one of the 63 target occurrences at $k = 10$ or $k = 25$. A positive control (forcing each mapped node to rank one) recovered 49 of 49, so the evaluator was sound and the zero was a real property of the retrieval design.

1.2 Why V2 was needed

The failure was not that retrieval found irrelevant material. The top results were topically right: problem, task and claim nodes that paraphrase the question. What was missing was the answer *identity*. The main report’s rank inspection makes this concrete: for question Q1, the widely known potentials M3GNet, CHGNet, NequIP and MACE sat at flat ranks 880, 1,340, 1,536 and 5,437 out of 5,480 documents. [MEASURED] (main report)

Figure 1 shows why. A long question is written in the vocabulary of problems, requirements and outcomes. A claim or task node is written in the same vocabulary. A method node is usually a short proper name. A single-vector similarity therefore ranks the texts that *describe* a solution above the name of the solution.

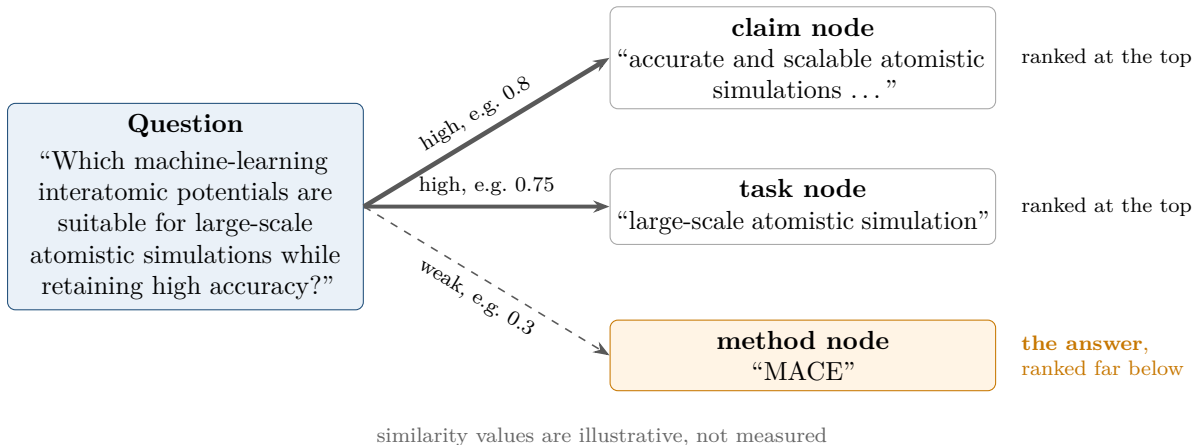


Figure 1: Why single-vector retrieval misses the answer identity. A long question shares its wording with claims and tasks, which describe the problem, far more than with a short method name such as “MACE”. Arrow thickness and the numbers are a schematic of cosine similarity, labelled illustrative because they were not measured; the measured consequence is the main report’s rank inspection (MACE at flat rank 5,437 of 5,480 for Q1). The texts that rank highest are evidence *about* an answer, not the answer.

The observation that motivates V2

Semantic-region retrieval $\neq$ answer-identity retrieval.

The remedy is not a better similarity function applied to the same objects. It is to change what is ranked and how evidence reaches it:

1. rank *answer entities*, not arbitrary graph nodes;
2. let the informative texts (claims, tasks, problems) act as *evidence that points to* an entity instead of competing with it;
3. separate the cheap step that decides which entities are considered at all from the expensive step that orders them.

The third point is the conceptual centre of this report. V2 separates **candidate admission**, which must find a high-recall set of plausible answer entities, from **candidate ordering**, which carefully reranks a much smaller set using joint question–document interaction.

1.3 V2 is a supplement, not a replacement

V2 improves the *extrinsic retrieval component* only. It does not replace or redefine the TSHC objective, the hierarchy construction, P1 laminarity, P2 budgets, P3 semantic coherence, P4 native hyperedge collapse, P5 temporal regularisation and persistent identities, or P6 label generation and held-out faithfulness evaluation. Those are established by the main `clean_tshc` system. V2 reads the same supplied TKH and answers the same benchmark questions (Figure 2).

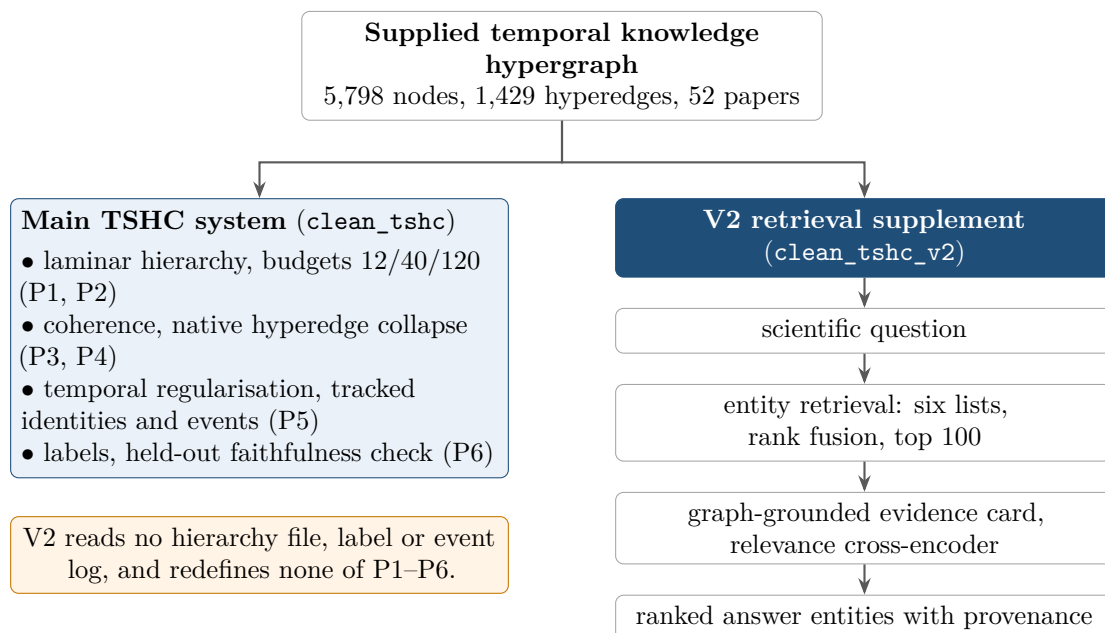


Figure 2: How the two packages relate. Both consume the same supplied hypergraph. The main system produces the hierarchy and everything the assessment’s properties P1–P6 refer to. V2 is a separate downstream branch that answers the benchmark questions by retrieving entities; it never reads the hierarchy’s outputs, so it cannot alter them and cannot be read as a competing clustering method.

What V2’s results cannot show

Because V2 does not traverse the hierarchy, a gain in V2 retrieval is *not* evidence that the hierarchy helps retrieval. The main report’s finding on that question (research question RQ4: not demonstrated) stands unchanged.

1.4 Contributions of V2

- **Canonical answer entities** instead of raw graph nodes, with conservative identity grouping and graph-supplied aliases (Section 4).
- **Six complementary first-stage lists**: two dense views, BM25, safe name matching, evidence-to-entity retrieval and bounded graph expansion (Section 7).
- **Evidence-to-entity retrieval**, which lets the texts that used to outrank the answer vote for it instead (Section 7.3).
- **Deterministic rank fusion** without score calibration or learned weights (Section 8).
- **Query-dependent candidate evidence**: an identity-first card of selected, provenance-bearing evidence (Section 11).
- **A genuine relevance cross-encoder** applied to the first 100 fused candidates (Section 10).
- **A strict prediction/evaluation firewall**: predictions are hashed and frozen before the scorer can read the ground truth (Section 15).

1.5 How to read this document

Statements are tagged where the distinction matters:

[GUARANTEED]

Holds by construction of the algorithm or is enforced by code that fails loudly otherwise.

[MEASURED]

A number read from a frozen artifact of the run described in Section 20.

[DESIGN CHOICE]

A fixed, declared setting chosen by judgement, not derived from data.

[ASSUMED]

Something the method relies on but that was not verified.

[DEVELOPMENT OBSERVATION]

A pattern seen on the development benchmark, which may not generalise.

“A6” is the project’s internal name for the configuration described here: the fused candidate list reranked by the relevance cross-encoder, with a reranking pool of $K = 100$. That depth was selected after development results were available (Figure 9).

2 Relation to the Assessment Requirements

This section states exactly where each assessment requirement is satisfied, so that the narrow scope of this supplement is not mistaken for an omission.

2.1 Inherited guarantees: P1–P6

The assessment defines six properties of a multi-resolution abstraction:

- P1 Laminar refinement**: each level refines the one above it.
- P2 Size budget**: level k has at most N_k super-nodes; a hard constraint.
- P3 Semantic coherence**: members of a super-node are close in meaning.
- P4 Hyperedge fidelity**: the method works on the hypergraph and its coarsening rule is the one actually applied.

- P5 Temporal stability:** the hierarchy evolves smoothly and identities are trackable.
- P6 Interpretable, faithful labels:** labels assert nothing the members, visible at the cutoff, do not support.

All six are properties of a *hierarchy*. V2 builds no hierarchy, so P1–P6 belong entirely to the main system and V2 neither weakens nor alters them (Table 1). What V2 shares with them is discipline: it consumes the same temporally valid graph data, keeps hyperedge provenance on every piece of evidence, and does not reinterpret citation or structural context as scientific entailment.

Table 1: Properties P1–P6: where each is established and what V2 has to do with it. The middle column summarises the main report; nothing in it was re-run here.

Requirement	Main TSHC responsibility	V2 relevance
P1 Laminar refinement	Guaranteed by agglomerative, merge-only construction; tested	Unchanged. V2 builds no partition; entity grouping is not a multi-level hierarchy.
P2 Size budget	Fixed budgets of 12, 40 and 120 super-nodes; tested	Unchanged.
P3 Semantic coherence	Hierarchy objective; evaluated with an independent encoder against null partitions	Unchanged. Rank fusion combines retrieval signals; it is not a coherence measurement.
P4 Hyperedge fidelity	Native, multiplicity-preserving hyperedge collapse used by the objective	V2 uses graph-derived evidence and provenance read from native hyperedges; it does not redefine or perform the collapse.
P5 Temporal stability	Variation-of-information regularisation, Hungarian identity matching, event log	Snapshot validity is retained: the same visibility rule filters nodes and hyperedges. No new stability claim.
P6 Faithful labels	Label and gloss generation with held-out NLI checks	No change and no labels. V2 separately respects the temporal visibility of the evidence it shows.

2.2 Tasks T1–T7

Table 2 does the same for the seven tasks. T6 is split into its six evaluation requirements, because V2 changes exactly one of them: the extrinsic retrieval experiment.

2.3 Scoring criteria

The assessment weights six criteria. Table 3 lists them with their weights and shows which document carries the evidence for each.

3 Retrieval Problem Formulation

3.1 Objects

Let $H_t = (V_t, E_t)$ be the hypergraph snapshot visible at year t (Section 14). A question is a string q . The retriever does not rank nodes; it ranks *canonical entities*.

Definition 3.1 (Retrieval task). *Let $\mathcal{E}_t$ be the set of canonical entities of H_t (Definition 4.1), let $\mathcal{E}_t(q) \subseteq \mathcal{E}_t$ be the entities eligible for q (Definition 5.1), and let $\text{Ev}_t(e)$ be the graph-grounded evidence items linked to entity e . The task is to produce an ordering*

$$\pi_q = (e_1, e_2, \dots, e_n), \quad e_i \in \mathcal{E}_t(q), \quad (1)$$

Table 2: Tasks T1–T7. The six evaluation requirements inside T6 are listed separately because only the last one is changed by V2.

Task	Covered in the main report by	This supplement
T1 Evolving graph and data description	Per-snapshot counts, type and arity distributions, growth, data anomalies	Uses the same file and visibility rule; reports the retrieval index built from it (Table 5).
T2 Formal method	Objective, structure/semantics trade-off, complexity, guarantees	Formal statement of the retrieval problem and ranking rule (Section 3 and Section 12), for the retrieval component only.
T3 Temporal coupling	Regularisation toward the previous snapshot; tracked identities	Unchanged. V2 applies one visibility cut-off per question (Section 14).
T4 Hyperedge collapse	Collapse rule designed, implemented and used	Unchanged. V2 reads hyperedges with role checks; it coarsens nothing.
T5 Labels	Generated labels and glosses; measured faithfulness	Unchanged.
T6 Coherence circularity issue	Different encoder for construction and for evaluation	Not applicable to V2 metrics, which are identity matches, not similarity scores. The analogous hazard for retrieval is leakage (Section 15).
T6 Independent coherence evaluation	BGE-based coherence at three levels	Unchanged.
T6 Null models	100 type- and size-preserving null partitions	Unchanged.
T6 Perturbation stability and confidence intervals	10% hyperedge deletion, five seeds, t -intervals; real cross-snapshot ARI	Unchanged. V2 reports point values only (Section 21).
T6 Label overclaim evaluation	NLI check against held-out members	Unchanged.
T6 Extrinsic retrieval	Coarse-to-fine against flat; negative result reported	Full detail in this document: architecture, isolation, protocol, results.
T7 Write-up; verified against assumed	Table of verified and assumed statements	Tags throughout and Table 16.

in which scientifically appropriate answer identities occur near the top, using only q , H_t and frozen pretrained models.

3.2 Two stages, two metrics

The system factorises as

$$q \xrightarrow{\text{admission}} P_K(q) \xrightarrow{\text{ordering}} \pi_q. \quad (2)$$

The stages have different goals and are measured differently. Let $\mathcal{T}$ be the multiset of target occurrences (one per expected method per question) and $\mathcal{T}_{\text{dir}} \subseteq \mathcal{T}$ those mapped directly to graph identities (Section 16). Write $M(\tau)$ for the set of entities a mapped target τ resolves to.

Definition 3.2 (Candidate recall).

$$\text{CandidateRecall@}K = \frac{|\{\tau \in \mathcal{T}_{\text{dir}} : M(\tau) \cap P_K(q_\tau) \neq \emptyset\}|}{|\mathcal{T}_{\text{dir}}|}. \quad (3)$$

Membership is measured on the pool before reranking. The denominator is the directly mapped occurrences (49 in the frozen evaluation).

Table 3: The assessment’s scoring criteria and where each is addressed. The supplement adds to the evaluation-validity and reproducibility evidence for the extrinsic task; the hierarchy-specific criteria rest on the main report.

Criterion	Weight	Main report	This supplement
Problem formulation and method design	25%	Objective, trade-off, guarantees against empirical properties	Retrieval problem formalised; each component’s purpose stated (Section 3–Section 11)
Evaluation validity	30%	Independent encoder, nulls, intervals, held-out labels, frozen predictions	Firewall and freeze (Section 15), denominators and ceilings (Section 16), development label (Section 17)
Temporal treatment	15%	Cross-snapshot stability and identity tracking	Visibility applied to nodes, edges and aliases (Section 14); no new stability claim
Hyperedge collapse	10%	Rule and implementation (T4)	Not changed; hyperedges supply role-checked evidence (Section 14)
Reproducibility	10%	Commands, pinned dependencies, reruns	Pinned revisions, hashes, replay, tests (Section 20)
Clarity; verified against assumed	10%	Verified/assumed table	Tags, caution boxes, Table 16

Table 4: Notation used in this report.

Symbol	Meaning
q, q_o, q_s	Question; its verbatim (original) view; its structured view
$H_t = (V_t, E_t)$	Snapshot visible at year t
$\mathcal{E}_t, \mathcal{E}_t(q)$	Canonical entities; those eligible for q
$\text{Ev}_t(e)$	Evidence items linked to entity e
$\phi(\cdot) \in \mathbb{R}^{384}$	Unit-norm embedding from the frozen bi-encoder
$L_c, r_c(e)$	Ranked list of channel c ; rank of e in it (from 1)
F_q	Fused order of the union of all lists
$P_K(q)$	Candidate pool: the first K entities of F_q ; here $K = 100$
$s_{\text{ce}}(q, e)$	Cross-encoder relevance logit
π_q	Final ranking

Definition 3.3 (Final recall). *With $\text{rank}_\pi(\tau) = \min_{e \in M(\tau)} \text{rank}_\pi(e)$,*

$$\text{R@}k = \frac{|\{\tau \in \mathcal{T}_{\text{dir}} : \text{rank}_\pi(\tau) \leq k\}|}{|\mathcal{T}|}. \quad (4)$$

The numerator counts only directly mapped targets; the denominator is all target occurrences (63), so targets that cannot be mapped count as misses.

The two denominators differ on purpose. Candidate recall asks “of the answers that exist in the graph, how many did stage one let through?”. Final recall asks “of everything the benchmark expects, how much did the system return?”. Both conventions are those of the frozen evaluator.

Proposition 3.1 (Admission is a ceiling). *Suppose the second stage only permutes $P_K(q)$ within the first K positions and leaves every other entity in fused order. Then for every $k \leq K$ the number of targets recovered at k is at most the number admitted to the pool, and at $k = K$ the two are equal, whatever scores the reranker assigns.*

Proof. A target τ has $\text{rank}_\pi(\tau) \leq K$ exactly when some $e \in M(\tau)$ occupies one of the first K positions. Those positions hold precisely the entities of $P_K(q)$, in some order. Hence $\text{rank}_\pi(\tau) \leq K$ iff $M(\tau) \cap P_K(q) \neq \emptyset$, and for $k < K$ the condition can only be harder to meet. $\square$

Candidate admission is a ceiling

A reranker cannot recover an entity that never enters its candidate pool. With $K = 100$, $R@100$ is fixed by the first stage alone [GUARANTEED]; the cross-encoder can only change which admitted entities appear earlier.

This is why the two stages are logically distinct. Admission should be tolerant, because its mistakes are final. Ordering should be discriminating, because it is the only place where a precise judgement is affordable.

4 Canonical Entity Representation

4.1 Why raw nodes are the wrong unit

The graph often stores one scientific object as several nodes. A method can appear as a `method` node extracted from the paper that proposes it, as a `cited_work` node from papers that cite it, and as a `component` node from a paper that uses it. Ranking nodes would list the same answer several times, split its evidence, and make the evaluation depend on which duplicate happens to be scored. V2 therefore groups nodes into canonical entities before anything is ranked.

4.2 Construction

Definition 4.1 (Canonical entity). *Let ν be surface normalisation: Unicode NFKC, case-folding, mapping typographic hyphens and the minus sign to “-”, and collapsing whitespace. Let the family map send the types `method`, `technique`, `component` and `cited_work` to the single family approach, and each of `dataset`, `metric` and `article` to itself. Two visible nodes u, v of these seven types are equivalent when they lie in the same family and $\nu(\text{surface}(u)) = \nu(\text{surface}(v))$. A canonical entity is an equivalence class. Its identifier is formed from the lexicographically smallest member node id; it records all member node ids, the set of member types, every distinct surface form, the graph-provided aliases, the earliest `first_seen_year` among members and each member’s provenance.*

Three properties of this definition are deliberate. [DESIGN CHOICE]

- **Exact, not fuzzy.** Grouping needs equal normalised surface forms. No edit distance, embedding similarity or abbreviation expansion is used, because a wrong merge silently fuses two scientific objects.
- **Compatible family.** A dataset and a method that share a name stay separate entities. In the frozen index “Materials Project” is both an approach entity and a dataset entity. [MEASURED]
- **Aliases help retrieval but do not merge.** An alias stored in the graph is searchable text for the name and lexical channels. It never causes two entities with different surface forms to be joined.

Claims, tasks and problems are not answer entities; they are evidence (Section 4.3). Authors and future-topic nodes are neither.

4.3 Evidence links

Identity and evidence are stored apart. An *evidence item* is a visible node of type claim, task, problem, technique, component, dataset, metric or article; its text is its surface form. An item

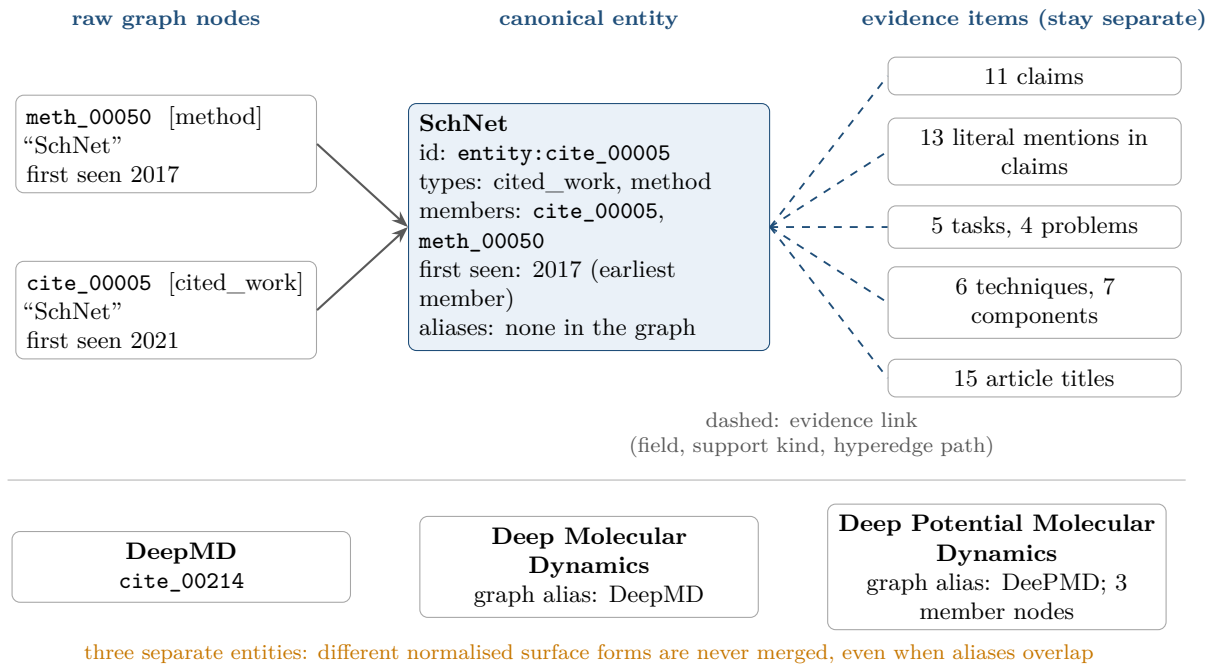


Figure 3: Canonical entity construction, with real identifiers from the frozen 2026 index. Top: the two nodes whose surface form normalises to “schnet” and whose types fall in the same family become one entity; its evidence remains a set of separate, linked nodes and is never folded into the identity. Bottom: three entities that a reader may regard as the same method stay separate because their surface forms differ. This is the price of refusing fuzzy merges, and it matters for evaluation (Section 18).

is *linked* to an entity only through a schema-checked hyperedge role, never through arbitrary co-membership **[GUARANTEED]**:

- **Typed method target.** A hyperedge of relation *addresses*, *solves*, *uses_technique*, *uses_component* or *evaluated_on* with exactly one *method* member and otherwise only members of the expected target type links each target to that method.
- **Explicit method claim.** A *claims* hyperedge with exactly one article, one claim and at most one method links the claim to the method, and to the asserting article.
- **Safe literal mention.** A claim whose text contains an entity’s safe name or alias as a whole token (Section 7.4) is linked as an explicit mention.
- **Article context.** The title of a presenting or provenance article is linked as context and flagged as *not* candidate-specific.

A hyperedge whose members do not fit its relation’s role pattern is rejected rather than guessed; the frozen index records zero rejections. Every link keeps the hyperedge id, relation, year and provenance of the path that justifies it.

Evidence coverage is thin

Only 470 of 2,978 entities have any candidate-specific evidence; the other 2,508 have article titles only. **[MEASURED]** Later stages can only be as informative as what is attached here.

5 Answer-Family Filtering

A question usually names the *kind* of thing it wants: “Which *methods* ...”, “What *datasets and benchmarks* ...”. V2 reads this answer head with a fixed pattern and restricts the candidate

Table 5: The frozen 2026 retrieval index. **[MEASURED]** Counts read from the frozen entity index file (Appendix B).

Item	Count	Item	Count
Nodes of the seven entity types	3,246	Evidence items	4,274
Canonical entities	2,978	Entity-evidence links	10,241
approach family	2,577	typed method target	3,903
datasets	259	article title (context)	3,550
metrics	90	safe literal mention	1,620
articles	52	article asserts claim	690
Entities with more than one node	240	explicit method claim	478
Entities with graph aliases	80	Directed role arcs	6,368
Entities with candidate-specific evidence	470	Rejected hyperedges	0

universe accordingly.

Definition 5.1 (Eligibility). *Let $A(q)$ be the answer heads recognised in q and let $\text{Fam}(a)$ be the node types configured for head a (Table 6). With $\text{Fam}(q) = \bigcup_{a \in A(q)} \text{Fam}(a)$, an entity e is eligible, $e \in \mathcal{E}_t(q)$, iff $\text{Fam}(q) = \emptyset$ or $\text{types}(e) \cap \text{Fam}(q) \neq \emptyset$.*

The filter is *hard*: an ineligible entity is removed from every channel and can never be ranked. **[GUARANTEED]** When no head is recognised, nothing is filtered.

Table 6: Answer heads and the node types they admit, from `config.json`. **[DESIGN CHOICE]** A coordinated head such as “datasets and benchmarks” takes the union of both rows.

Answer head in the question	Admitted node types
method, model, architecture, algorithm, tool, system	method, technique, component, cited_work
technique	technique, method, component
dataset	dataset
benchmark	dataset, metric
metric	metric
article, paper	article, cited_work

The breadth of the first row is necessary, not generous. Several well-known potentials exist in this graph *only* as cited works: the entities named MACE and GAP each consist of a single `cited_work` node. **[MEASURED]** A filter that admitted only `method` nodes would delete them. For the thirteen Type-A questions whose head is “methods” or “architectures”, 2,577 entities are eligible; for the one “datasets and benchmarks” question, 349. **[MEASURED]**

A typing mismatch can remove a valid answer

The filter trusts the graph’s node types. When the graph types an entity differently from how the question’s author thinks of it, a scientifically valid target is excluded before any channel runs. This happened in the frozen run: for the dataset question, the expected answers JARVIS-DFT, AFLOW and Alexandria exist in the graph, but as a `component`, a `cited_work` and a `cited_work/method` entity respectively, so none is eligible under the family `{dataset, metric}`. **[MEASURED]** These three are the only directly mapped targets that never enter the candidate union.

6 Two Query Views

Each question is represented twice.

- The **original view** q_o is the question string, unchanged. The code asserts that it is byte-identical to the input. **[GUARANTEED]**
- The **structured view** q_s is produced by a deterministic, purely extractive parser: regular expressions locate the answer head, the core task, “when/with/without” clauses and a temporal phrase, and copy those spans. The parser consults no model, no domain vocabulary and no answer data, and it never introduces a word that is not in the question. **[GUARANTEED]**

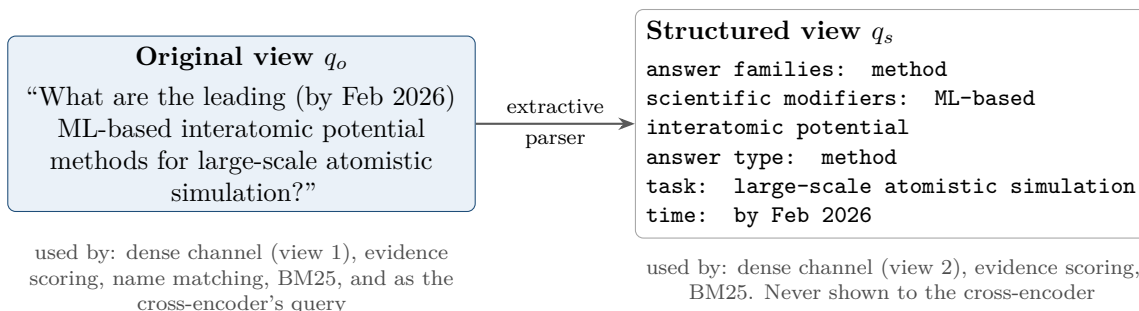


Figure 4: The two views of benchmark question Q12, exactly as stored in the frozen `query_understanding` trace. The structured view removes interrogative scaffolding and isolates the task and modifiers; it contains only words copied from the question and no candidate names. The original view is always preserved and is the only text the cross-encoder sees as the query.

The two views retrieve different things. The original view carries the full phrasing, including conditions the parser may not separate. The structured view concentrates the embedding on the task and its modifiers, which moves it toward task- and technique-shaped evidence. Neither replaces the other: they feed separate dense lists, and for evidence scoring an item takes the larger of its two similarities.

The parser is a heuristic. **[ASSUMED]** It can split a compound phrase imperfectly, and it does not separate every modifier. That is acceptable only because the structured view is *supplementary*: the verbatim question is always present in parallel.

7 First-Stage Candidate Generation

Six ranked lists are produced for each question. Each list answers a different question about a candidate, and each fails in a different way.

7.1 Dense semantic retrieval

Model. BAAI/bge-small-en-v1.5 (Xiao et al., 2023), pinned at revision 5c38ec7c405e (full hash in Table 18), run on CPU with weights loaded from local files whose SHA-256 hashes are checked before prediction starts. It is a *bi-encoder* in the Sentence-BERT sense (Reimers and Gurevych, 2019): one transformer maps any text to a single vector.

How it scores. Every non-author node v has a document: its own “[type] surface form” line followed by at most 12 deduplicated direct neighbours through content relations, claims first. This is the main report’s document builder, reused unchanged. The encoder gives unit-norm

vectors $\phi(\cdot) \in \mathbb{R}^{384}$, so cosine similarity is an inner product. The score of an entity is the best score of its member nodes:

$$s_d^x(e) = \max_{v \in \text{nodes}(e)} \langle \phi(q_x), \phi(\text{doc}(v)) \rangle, \quad x \in \{o, s\}. \quad (5)$$

The two views give two separate lists; they are not averaged. The code passes each view to the encoder as plain text and adds no instruction prefix of its own.

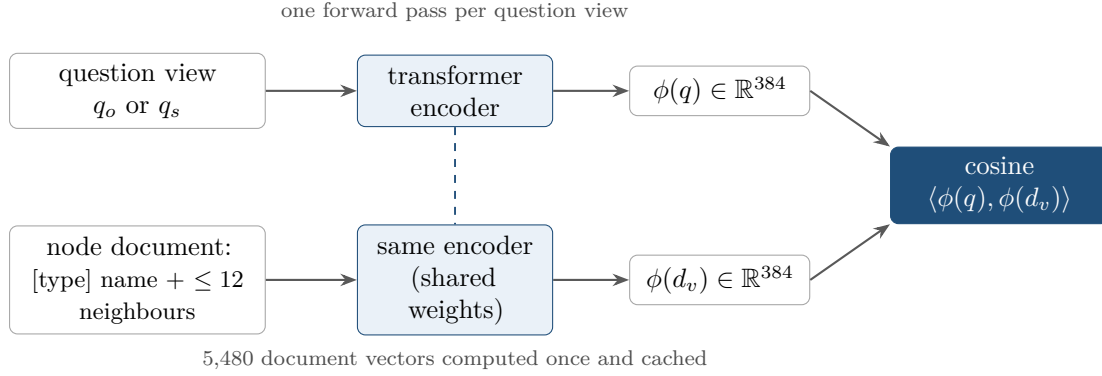


Figure 5: The bi-encoder used by the dense and evidence channels. Question and document pass through the same network *separately*; each is compressed to one 384-dimensional vector, and the only interaction between them is a final inner product. Because document vectors do not depend on the question they are computed once, which makes scoring every candidate cheap. The cost of that speed is that no question token ever attends to a document token.

Why it is fast. Document vectors are independent of the question. They are encoded once and cached under a content hash; a question then costs one encoder pass per view plus a matrix product.

Why it is imperfect. All of a text’s meaning must fit one vector, and the question and the document never interact token by token. A short name carries little signal, and a document whose *context* resembles the question can outrank the entity the question is about. This is the failure of Figure 1, and the reason dense retrieval is one channel among six rather than the retriever.

7.2 BM25 lexical retrieval

BM25 (Robertson and Zaragoza, 2009) scores exact term overlap. Each entity gets one lexical document: its names, its aliases and the text of every evidence item linked to it, each item once. Text is case-folded and split into word tokens, keeping internal hyphens and dots so that a token such as `e3-equivariant` survives intact. For a query Q (the original and the structured view concatenated) and entity document d :

$$\text{BM25}(Q, d) = \sum_{w \in Q} \underbrace{\ln\left(1 + \frac{N - n_w + 0.5}{n_w + 0.5}\right)}_{\text{IDF}(w)} \cdot \frac{f_{w,d} (k_1 + 1)}{f_{w,d} + k_1 (1 - b + b |d|/|d|)}, \quad (6)$$

with $k_1 = 1.5$ and $b = 0.75$ [DESIGN CHOICE]. The three ingredients are:

- **Term frequency** $f_{w,d}$: how often word w occurs in d . The fraction saturates, so the tenth occurrence adds far less than the first; k_1 sets how quickly.
- **Inverse document frequency**: n_w of the N documents contain w . A word found in few documents is strong evidence; a word found everywhere is nearly worthless.

- **Length normalisation:** $|d|/\sqrt{|d|}$ compares the document’s length with the average, so a long document is not favoured merely for containing more words; b sets how strongly.

Each distinct query word is counted once, and an entity with no overlapping word gets no entry at all, so it receives no vote in fusion. **[GUARANTEED]**

Toy example. Take $N = 1000$ entity documents and the query words “equivariant” ($n_w = 20$) and “potential” ($n_w = 600$). Then $\text{IDF}(\text{equivariant}) = \ln(1 + 980.5/20.5) \approx 3.89$ and $\text{IDF}(\text{potential}) = \ln(1 + 400.5/600.5) \approx 0.51$. For a document of average length containing each word once, the second factor is $2.5/2.5 = 1$, so the rare technical term contributes about 7.6 times as much as the common one. This is the behaviour one wants for scientific text.

Why it complements dense retrieval. An embedding smooths meaning; BM25 does not. It is valuable precisely where smoothing hurts: technical terms, rare abbreviations, exact terminology, names and aliases, and numbers or domain-specific tokens that a general-purpose encoder may represent poorly. Conversely it is blind to paraphrase, which the dense channels handle. **[ASSUMED]** (complementarity is a design premise; the measured channel coverage is in Table 8).

7.3 Evidence-level dense retrieval

This channel is the direct answer to Figure 1. Instead of asking whether the question resembles the entity’s name,

$$\text{question} \longrightarrow \text{“MACE”},$$

it asks whether the question resembles something the graph *says about* the entity, and then follows the link:

$$\text{question} \longrightarrow \text{relevant claim, task or other evidence} \longrightarrow \text{entity}.$$

Each evidence item i is embedded on its own, from its own text, with the same frozen encoder. Its similarity to the question is the better of the two views:

$$\text{sim}_q(i) = \max(\langle \phi(q_o), \phi(i) \rangle, \langle \phi(q_s), \phi(i) \rangle). \quad (7)$$

Let $S_{500}(q)$ be the 500 items with the highest sim_q (the *seeds*). An entity inherits the score of its best linked seed:

$$s_{\text{ev}}(e) = \max_{i \in \text{Ev}_t(e) \cap S_{500}(q)} \text{sim}_q(i). \quad (8)$$

Entities owning no seed are absent from this list.

Why max pooling. The alternatives are a sum or a mean over an entity’s evidence. A sum rewards entities for having *many* linked items, which favours heavily cited or heavily described entities regardless of the question. A mean punishes an entity for also having evidence about unrelated matters. The maximum asks the question that matters for admission: is there *at least one* piece of evidence strongly related to the question? It is also invariant to how much unrelated evidence an entity carries. **[DESIGN CHOICE]**

7.4 Safe name and alias channel

This channel has a narrow purpose: if a question literally names an entity, that entity should be admitted. An entity enters the list when one of its names or graph aliases occurs in the original question as a whole token sequence, and only if the name is *safe*: at least three characters long and either identifier-like (it contains a digit, a lower-case letter followed by an upper-case one, or

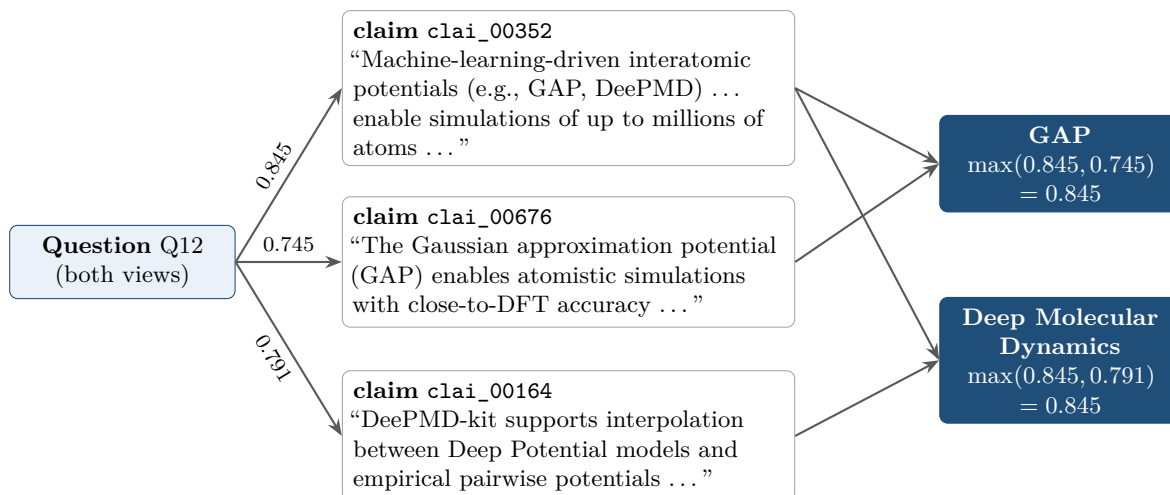


Figure 6: Evidence-to-entity retrieval, with similarities and links read from the frozen trace of question Q12. Each claim is embedded independently and compared with the question. A claim then passes its similarity to every entity it is linked to; here the links are safe literal mentions of the entity’s name or graph alias. One claim can support several entities, and an entity keeps the maximum over its linked seeds. The long, question-like claims that would have outranked a bare method name now vote for it. (Each entity has further seeds, omitted for clarity; the maximum shown is the logged channel score.)

is entirely upper case) or multi-word with at least ten characters. The score is the length of the longest match. The rule exists to stop short generic words from matching.

Its contribution should not be overstated. Benchmark questions describe a need; they do not name their answers. In the frozen run this list is empty for 9 of 18 questions and never longer than two entities, and it contains none of the 49 mapped targets. **[MEASURED]** What it matched were terms the question uses in passing, such as “DFT”. The same safe rule also decides which claims count as literal mentions (Section 4.3), and it is conservative: “CHGNet”, “MEGNet” and “Orb” are not identifier-like under it, so those entities receive no mention evidence. **[MEASURED]**

7.5 Bounded graph expansion

The last channel uses structure. Starting from the 200 most question-similar evidence items, it follows directed, role-checked arcs to eligible entities. Only the path shapes of Figure 7 exist.

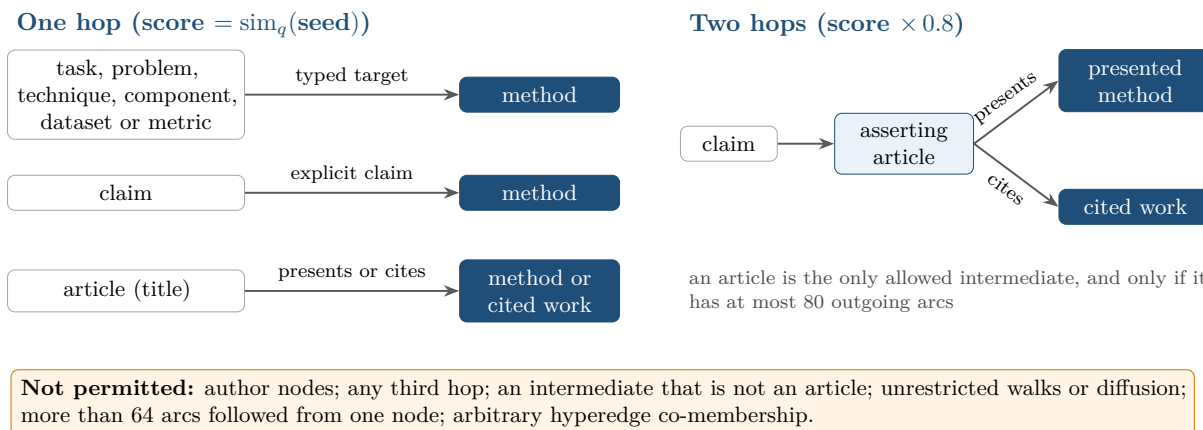


Figure 7: Every path shape the graph channel can follow, from the implementation. A seed is an evidence item similar to the question; arrows are directed arcs created only from hyperedges whose members fit the relation’s role pattern. A path’s score is the seed’s similarity multiplied by 0.8 for each hop beyond the first, and an entity keeps its best path. Depth, fan-out and hub size are all capped, and authors are never traversed. Each retained path stores its hyperedge ids, relations, years and provenance.

Algorithm 1 Bounded graph expansion**Input:** Seeds $(i, \text{sim}_q(i))$ sorted by similarity; arcs; eligible set $\mathcal{E}_t(q)$ **Output:** Graph score $s_{\text{gr}}(e)$ and up to 8 provenance paths per entity

```

1: for each of the first 200 seeds  $i$  do
2:   for each of the first 64 arcs  $i \rightarrow x$  do
3:     paths  $\leftarrow \{(i \rightarrow x)\}$ 
4:     if  $x$  is an article with at most 80 outgoing arcs then
5:       add  $(i \rightarrow x \rightarrow y)$  for each of the first 64 arcs  $x \rightarrow y$  with relation presents or
       cites,  $y \neq i$ 
6:     end if
7:     for each path  $p$  ending at node  $z$  of an eligible entity  $e$  do
8:        $s_{\text{gr}}(e) \leftarrow \max(s_{\text{gr}}(e), \text{sim}_q(i) \cdot 0.8^{|p|-1})$ 
9:       record  $p$  with its hyperedge ids, relations, years and provenance
10:    end for
11:  end for
12: end for
13: keep the 8 best paths per entity, ties broken by seed id

```

What it adds. An entity can be relevant without any evidence item of its own resembling the question. If a paper whose title or claims match the question *presents* or *cites* the entity, the graph channel can still admit it. For example, in the frozen Q12 trace the cited-work entity GAP is reached in one hop from a review article’s title (similarity 0.826, so score 0.826) and in two hops from a claim about DeePMD-kit through that claim’s article (similarity 0.791, so score $0.791 \times 0.8 = 0.633$). **[MEASURED]**

Structural context is not entailment

A path says that an entity is *near* question-relevant text in the graph. It does not say that the text is *about* the entity, and still less that the entity satisfies the question. The two-hop example above reaches GAP from a claim about a different method. Across all retained paths of the frozen run, 18,230 of 22,122 (82%) pass through a citation arc. **[MEASURED]** The implementation labels every such path `structural_path_not_entailment` and carries that label into the evidence card. The graph channel is an admission signal only.

7.6 Summary and measured coverage

Table 7 summarises the six lists. Table 8 then reports, for the frozen run, how many of the directly mapped targets each list reaches when used alone.

Table 8 supports the design argument of Figure 1 on this benchmark: the two dense lists, which compare the question with the entity’s own document, cover fewer than half of the mapped targets, while the evidence-owner list covers 44 of 49. **[DEVELOPMENT OBSERVATION]** The three targets no list covers are the three removed by the answer-family filter (Section 5).

8 Multi-Channel Union and Rank Fusion

8.1 Reciprocal rank fusion

The six lists must be combined into one order. V2 uses reciprocal rank fusion (Cormack et al., 2009):

Table 7: The six first-stage lists. Each is truncated to 500 entities before fusion.

List	Score	What it can find; how it fails
Dense, original	Equation 5 with q_o	Paraphrase of the full question; misled by short names and by context
Dense, structured	Equation 5 with q_s	Task- and modifier-focused paraphrase; inherits parser errors
BM25	Equation 6	Exact technical terms, names, aliases; blind to paraphrase
Safe name	length of longest match	An entity literally named in the question; usually empty
Evidence owner	Equation 8	Entities whose linked evidence matches the question; needs evidence to exist
Bounded graph	best path score	Entities presented or cited near matching text; structural, not semantic

Table 8: Coverage of the 49 directly mapped target occurrences by each list alone, within its 500-entity truncation. **[MEASURED]** “Unique” counts targets found by that list and by no other. Source: channel_diagnostics in the frozen metrics.json.

List	Targets covered	Fraction of 49	Unique
Dense, original view	19	0.388	0
Dense, structured view	21	0.429	0
BM25	41	0.837	0
Safe name	0	0.000	0
Evidence owner	44	0.898	0
Bounded graph	37	0.755	1
Union of all six	46	0.939	

Definition 8.1 (RRF score). Let $r_c(e) \in \{1, 2, \dots\}$ be the rank of entity e in list c . Then

$$\text{RRF}(e) = \sum_{c: e \in L_c} \frac{1}{k + r_c(e)}, \quad k = 60, \quad (9)$$

where a list that does not contain e contributes nothing. Entities are sorted by decreasing RRF, ties broken by entity id.

The constant k controls how much the very top of a list dominates. With $k = 60$, rank 1 contributes $1/61$ and rank 10 contributes $1/70$: close enough that agreement between several lists outweighs a single first place. $k = 60$ is the value proposed in the original paper and was fixed in advance. **[DESIGN CHOICE]**

RRF solves a calibration problem

The channels’ raw scores live on unrelated scales. In the frozen Q12 trace the best dense cosine is 0.835, the best BM25 score is 18.61, the single name score is 32, and graph scores are cosines multiplied by a decay. **[MEASURED]** Adding them would let BM25 or the name length dominate for no scientific reason; rescaling them would require weights, and weights would have to be fitted on the very benchmark being evaluated. A rank is comparable across channels without any fitting.

Rank fusion has costs. It discards magnitude: a channel that is very confident and one that barely prefers its top result contribute equally. And it rewards agreement, so an entity found

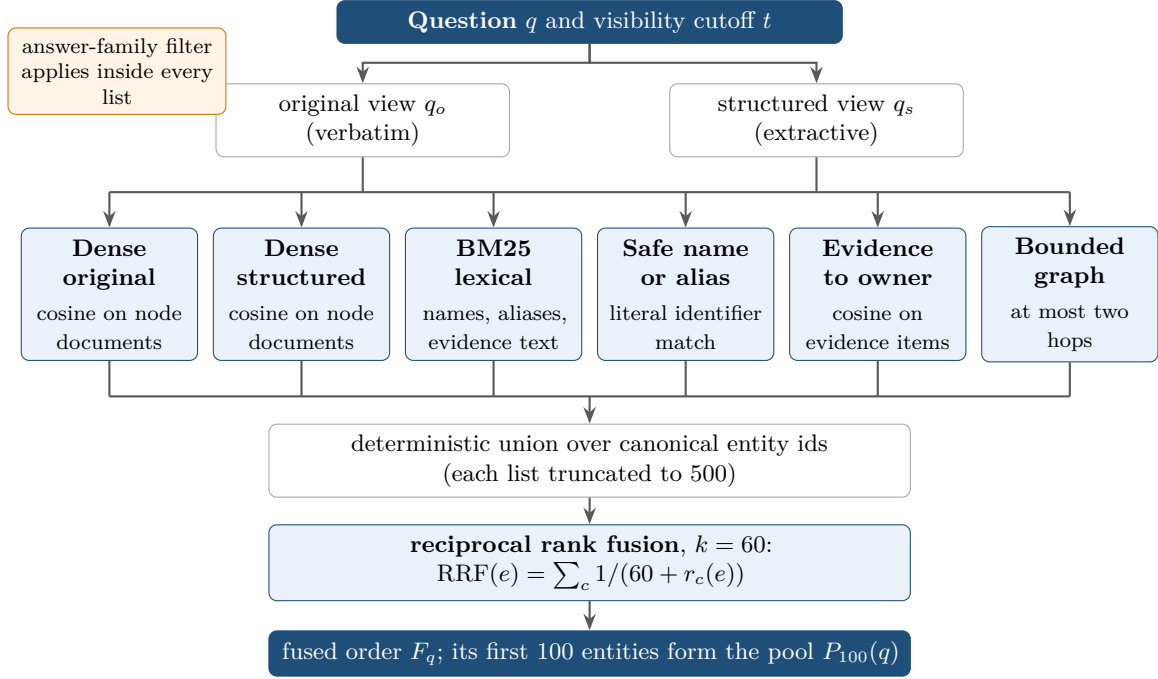


Figure 8: The first stage. One question becomes two views; six lists are computed independently, each restricted to eligible entities; their union is ordered by reciprocal rank fusion. Nothing in this stage is learned and nothing depends on run order: lists are sorted by score with ties broken by entity id, and the fusion sums are accumulated in a fixed channel order.

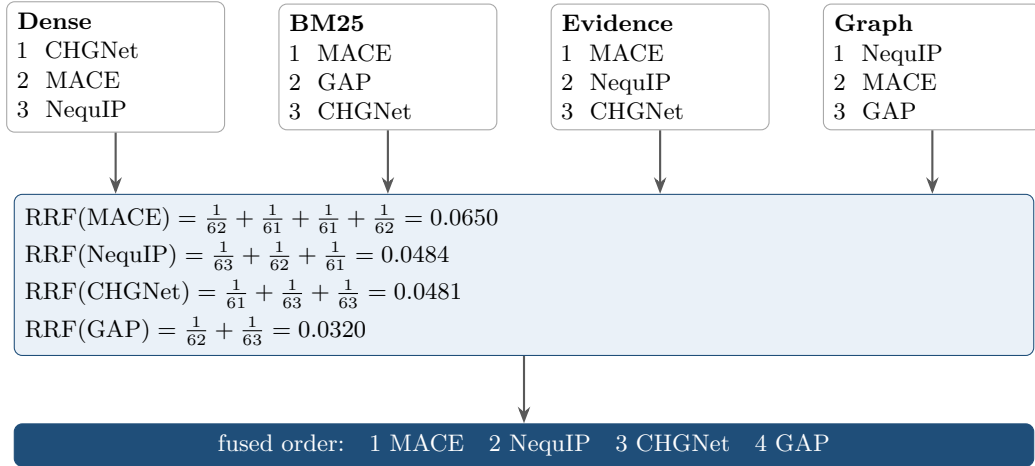


Figure 9: A toy fusion with four *fictional* rankings, constructed for teaching and not taken from any run. MACE is first in only two lists but present near the top of all four, so it wins clearly. NequIP and CHGNet each appear in three lists; NequIP edges ahead because its ranks (3, 2, 1) are slightly better than CHGNet’s (1, 3, 3). GAP appears in only two lists and comes last. Only ranks enter the computation; no raw score appears anywhere.

by one channel at rank 1 can be placed below an entity that several channels rank at 40. Both effects are visible in the worked example of Section 13.

9 Candidate Admission versus Ranking

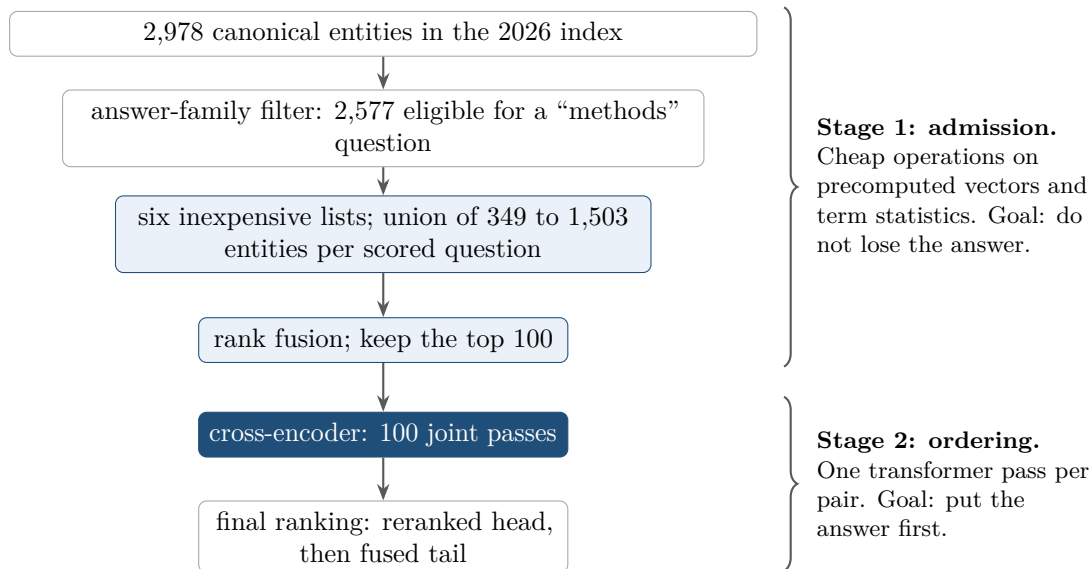


Figure 10: The two-stage funnel with counts from the frozen run. Stage one applies cheap operations to every eligible entity and is judged by recall. Stage two applies an expensive joint model to 100 candidates and is judged by ordering quality. Entities ranked below 100 by fusion are not discarded: they keep their fused order after the reranked head.

The split is first a matter of cost. A bi-encoder comparison is an inner product of two vectors that already exist. A cross-encoder comparison is a full transformer forward pass over the concatenated question and candidate text, and it cannot be precomputed because its input depends on the question. Running it over all 2,577 eligible entities would cost about 26 times as many forward passes per question as running it over 100. The two stages therefore optimise different things:

- **First stage: maximise reasonable recall.** False positives are tolerable, because the next stage will reorder them. False negatives are permanent (Proposition 3.1).
- **Second stage: maximise ordering quality.** It may spend a model’s full capacity on each pair, because there are few pairs.

The pool size K trades admission against the amount of plausible-but-wrong material the reranker must sort through.

How $K = 100$ was chosen

Among the predeclared reranking depths evaluated during development, $K = 100$ gave the strongest observed top-10 recovery. This supplement describes that simplified configuration. This selection was made after development results were available and is therefore a development choice, not an independently validated optimum. **[DEVELOPMENT OBSERVATION]** The original frozen run declared a deeper pool as its primary setting; $K = 100$ was one of its predeclared alternatives, not its primary configuration.

10 Cross-Encoder Reranking: the A6 Stage

What distinguishes A6 from its own first stage is one additional operation: the first 100 fused candidates are rescored by a model that reads the question and the candidate *together*.

10.1 Bi-encoder and cross-encoder

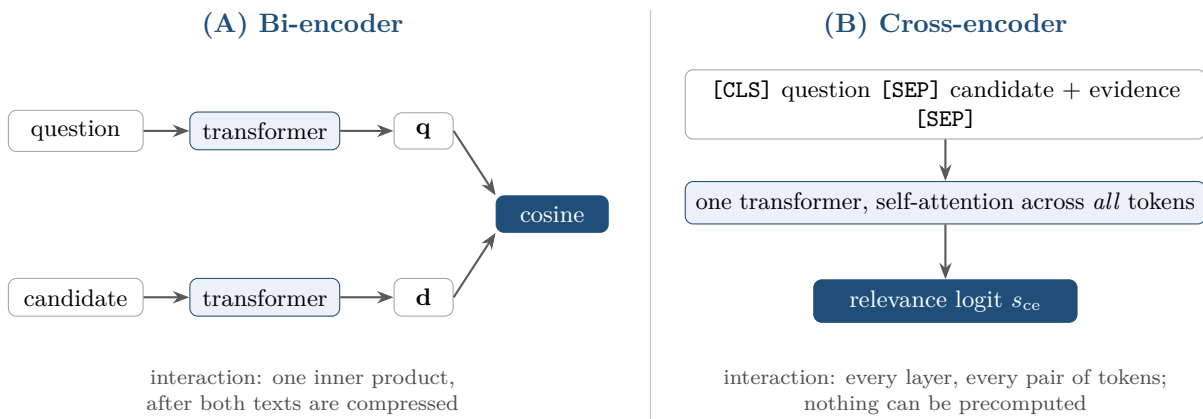


Figure 11: The two ways of scoring a question–candidate pair. (A) A bi-encoder encodes the two texts separately and compares two fixed vectors. (B) A cross-encoder concatenates the texts and lets every token attend to every other token before producing a single score. (A) is cheap and used to search everything; (B) is expensive and used on 100 candidates.

In a transformer layer (Vaswani et al., 2017) each token’s new representation is a weighted mixture of all tokens in the sequence:

$$h_i^{(\ell+1)} = g\left(h_i^{(\ell)}, \sum_j \alpha_{ij}^{(\ell)} h_j^{(\ell)}\right), \quad \alpha_{ij}^{(\ell)} = \text{softmax}_j(\langle W_Q h_i^{(\ell)}, W_K h_j^{(\ell)} \rangle / \sqrt{d}). \quad (10)$$

In a bi-encoder the sum runs over the tokens of *one* text. In a cross-encoder it runs over the concatenation, so the attention weight α_{ij} can connect a question token i to a candidate token j in every layer (Devlin et al., 2019; Nogueira and Cho, 2019). The model can therefore check whether a specific condition in the question is addressed by a specific phrase in the candidate’s evidence, which a single pooled vector cannot represent reliably.

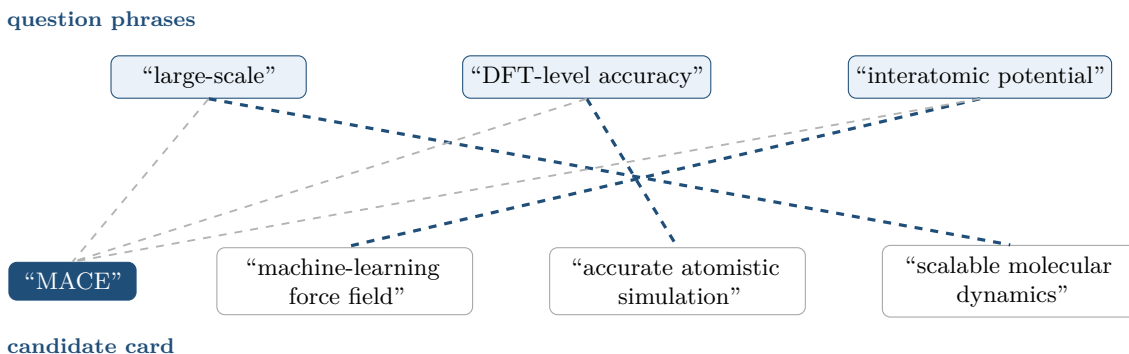


Figure 12: What joint attention makes possible, as a schematic. The question’s conditions (top) can be matched with the corresponding phrases in the candidate’s evidence (bottom): “large-scale” with “scalable molecular dynamics”, “DFT-level accuracy” with “accurate atomistic simulation”, “interatomic potential” with “machine-learning force field”. All of them connect to the identity “MACE”, which by itself shares no words with the question. The phrases and links are illustrative; attention weights were not measured.

10.2 The model actually used

The reranker is `cross-encoder/ms-marco-MiniLM-L6-v2`, pinned at revision `233902d25c44` (full hash in Table 18), loaded from local files with hash checks and run on CPU. It is a MiniLM-family transformer (Wang et al., 2020) with a single regression output, trained for passage relevance on MS MARCO web-search data (Bajaj et al., 2016). Its use here follows the standard retrieve-then-rerank pattern (Nogueira and Cho, 2019).

Why a cross-encoder?

Unlike a bi-encoder, query and candidate tokens interact in the same transformer. The question’s wording can be matched against the candidate’s evidence phrase by phrase, instead of comparing two summaries.

What the output is and is not

The model’s output is a **relevance logit**: an unbounded real number whose only meaningful use here is to order candidates of the same question. It is *not* an entailment or NLI judgement, *not* a statement of scientific truth, and *not* a calibrated probability. No sigmoid, threshold or cross-question comparison is applied. The loader refuses any model that has more than one output or NLI-style labels. **[GUARANTEED]**

10.3 The ranking rule

For each $e \in P_{100}(q)$ the model receives the original question q_o and the packed candidate card $\text{card}(e, q)$ of Section 11 and returns $s_{ce}(q, e) \in \mathbb{R}$. The final ranking is

$$\pi_q = \underbrace{\text{sort}(P_{100}(q); -s_{ce}, \text{entity id})}_{\text{reranked head}} \oplus \underbrace{F_q[101:]}_{\text{untouched fused tail}}. \quad (11)$$

The head is ordered by logit alone; the fused score is not mixed back in. The tail is exactly the fused order, so every entity of the union keeps a final rank. **[GUARANTEED]** The test `test_reranker_only_changes_frozen_prefix` checks this.

11 The Query-Dependent Candidate Card

A cross-encoder judges text. Given only “MACE”, it would face the problem of Figure 1 again. So each candidate is presented as a *card*: its identity followed by a bounded selection of its most question-relevant evidence.

11.1 What the card contains

The identity block is always first:

Candidate: <name> Type: <member types>

It is followed by evidence blocks drawn from ten families, visited in the fixed priority order of Table 9. Within a family, rows are sorted by decreasing question similarity sim_q of Equation 7, then candidate-specific rows before contextual ones, then by evidence id. Each block is rendered as <family> [<support kind>]: <evidence text>, so the model sees how the evidence is connected to the candidate.

Four rules govern the card. **[GUARANTEED]**

- **Identity is retained.** The identity block is never shortened or dropped. If the question and identity together could not fit, the *question* would be shortened first; this never happened in the frozen run.

Table 9: Evidence families of the card, in priority order, with the maximum number of blocks per family from `config.json`. [DESIGN CHOICE] At most 20 evidence blocks follow the identity.

	Family	Cap	How the evidence is attached to the candidate
1	claims	4	Explicit method claim, or a claim asserted by a candidate article
2	explicit mentions	3	Claim that literally contains the candidate’s safe name or alias
3	tasks	2	Typed target of an <code>addresses</code> hyperedge
4	problems	2	Typed target of a <code>solves</code> hyperedge
5	techniques	2	Typed target of a <code>uses_technique</code> hyperedge
6	components	2	Typed target of a <code>uses_component</code> hyperedge
7	datasets	1	Typed target of an <code>evaluated_on</code> hyperedge
8	metrics	1	Typed target of an <code>evaluated_on</code> hyperedge
9	articles	1	Title of a presenting or provenance article; context only
10	graph evidence	2	Seed text of a retained graph path; structural, not entailment

- **Evidence is deduplicated.** A text already placed on the card, compared after normalisation, is skipped in every later family.
- **Evidence is selected query-dependently.** The same entity shows different evidence for different questions, because selection is driven by sim_q .
- **Everything left out is logged.** Blocks dropped by a family cap are recorded with the reason `family_cap`; blocks dropped for length with `priority_prefix_token_limit`; the final token count is stored per pair.

11.2 Packing into 512 tokens

The model accepts at most 512 tokens for the whole input, including three special tokens. Blocks are appended in priority order until the next block would exceed the limit; that block and all after it are omitted (Algorithm 2). The rule keeps a *prefix* of the priority order, so a long high-priority block is never traded for a shorter low-priority one.

Measured card statistics. Over the 1,800 question–candidate pairs that make up the 100-candidate pools of all 18 questions, the packed input has between 54 and 512 tokens (median 176, mean 207.7). The median card has four blocks including the identity and the largest has 19. In 81 pairs at least one block was dropped by the token limit; in 994 at least one row was left out by a family cap; the question was never truncated. [MEASURED] (tabulated for this report from the frozen cross-encoder and evidence traces)

A card can be almost empty

Because most entities have only article titles attached (Table 5), many cards consist of an identity and one title. The cross-encoder then judges little more than a name and a paper title, and its score reflects that poverty of evidence rather than the entity’s merit.

12 The Complete A6 Algorithm

A note on how the frozen run computed this. The frozen run scored a deeper fused prefix once (configuration field `rerank_k` = 500) and stored several rankings derived from the same scores. The ranking defined by Algorithm 3 with $K = 100$ is stored under the key `A6_100`; depth 100 was a predeclared alternative to the run’s primary depth and was adopted for this supplement after the development results were available (Figure 9). A card and its logit depend

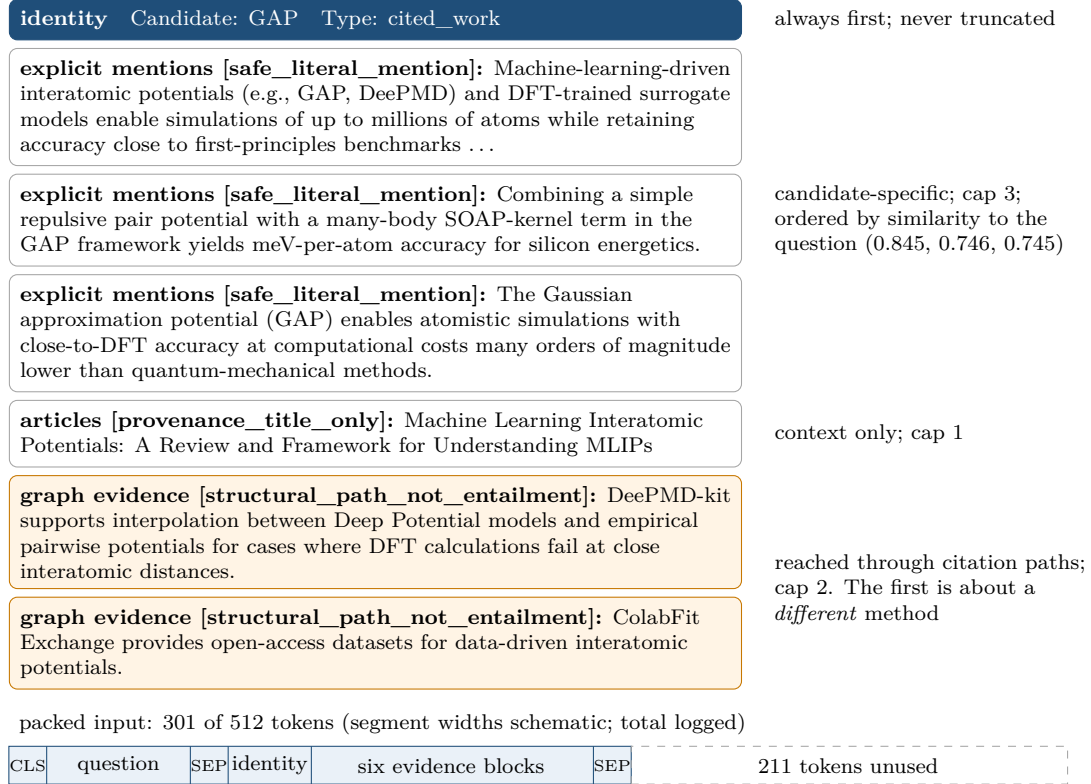


Figure 13: A real candidate card: the entity GAP for question Q12, as stored in the frozen cross-encoder trace (first block abridged). The card opens with the identity and continues with evidence in priority order; orange blocks are structural context that the card itself marks as not entailment. Thirty-eight further evidence rows were left out by family caps and logged. The bar shows the model input: the question, a separator, and the card, occupying 301 of the 512 available tokens. The cross-encoder returned a logit of +1.296 for this pair.

Algorithm 2 Card construction and token packing for one candidate

Input: Entity e ; similarities $\text{sim}_q(\cdot)$; retained graph paths of e ; caps; limit $L = 512$

Output: Model input (q' , document) and a log of included and omitted blocks

```

1: blocks  $\leftarrow$  [ identity( $e$ ) ]; seen  $\leftarrow \emptyset$ 
2: for each family in priority order (Table 9) do
3:   rows  $\leftarrow$  evidence linked to  $e$  in this family (for graph evidence: path seeds)
4:   sort rows by ( $-\text{sim}_q$ , candidate-specific first, evidence id)
5:   for each row with normalised text not in seen do
6:     add text to seen
7:     if fewer than cap blocks taken from this family then append block
8:     else log omission family_cap
9:     end if
10:  end for
11: end for
12:  $q' \leftarrow q_o$ ; if  $q_o$  leaves no room for the identity, truncate  $q'$  (never the identity)
13: document  $\leftarrow$  identity text
14: for each evidence block in order do
15:   if  $\text{tokens}(q', \text{document} + \text{block}) \leq L$  then document  $\leftarrow$  document + block
16:   else log this and all remaining blocks as priority_prefix_token_limit; break
17:   end if
18: end for
19: return ( $q'$ , document), included blocks, omitted blocks, token count

```

Algorithm 3 A6 retrieval**Input:** Questions; graph H ; frozen encoder ϕ ; frozen cross-encoder f ; fixed configuration**Output:** For each question a ranked list of canonical entities with full provenance; a freeze manifest

```

1: verify the SHA-256 of every pinned model file; fix seeds; set deterministic execution
2: for each distinct visibility cutoff  $t$  do
3:    $H_t \leftarrow$  visible snapshot; build entities, evidence links and arcs (Section 4)
4:   encode node documents and evidence items with  $\phi$  (cached by content hash)
5: end for
6: for each question  $q$  do
7:   parse answer heads  $A(q)$  and cutoff; build  $q_o$  (verbatim) and  $q_s$  (extractive)
8:    $\mathcal{E}_t(q) \leftarrow$  entities whose types meet the answer families (all, if no head)
9:   compute  $\text{sim}_q(i)$  for every evidence item  $i$  ▷ Equation 7
10:   $L_{do}, L_{ds} \leftarrow$  dense lists for  $q_o$  and  $q_s$  ▷ Equation 5
11:   $L_{bm} \leftarrow$  BM25 on  $q_o \oplus q_s$ , positive scores only ▷ Equation 6
12:   $L_{nm} \leftarrow$  safe name and alias matches in  $q_o$ 
13:   $L_{ev} \leftarrow$  owners of the 500 best evidence seeds ▷ Equation 8
14:   $L_{gr} \leftarrow$  bounded expansion from the 200 best seeds ▷ Algorithm 1
15:  truncate each list to 500;  $F_q \leftarrow$  union sorted by  $(- \text{RRF}, \text{entity id})$  ▷ Equation 9
16:   $P \leftarrow$  first  $K = 100$  entities of  $F_q$ 
17:  for each  $e \in P$  do
18:    build and pack  $\text{card}(e, q)$  within 512 tokens ▷ Algorithm 2
19:     $s_e \leftarrow f(q_o, \text{card}(e, q))$  ▷ relevance logit
20:  end for
21:   $\pi_q \leftarrow P$  sorted by  $(-s_e, \text{entity id})$ , followed by  $F_q[101:]$  unchanged ▷ Equation 11
22:  write  $\pi_q$ , per-channel ranks and scores, selected and omitted evidence, token counts
23: end for
24: write predictions; compute their SHA-256; write the freeze manifest

```

only on the question and the candidate, not on the pool depth, so sorting the first 100 fused candidates by those stored logits is the same computation as Algorithm 3; the frozen file confirms that this ranking’s first 100 entries are a permutation of the fused top 100 and that every later entry is in fused order. **[MEASURED]**

13 Worked End-to-End Example

This section follows one benchmark question through the whole system. Every candidate name, rank and score below is read from the frozen *prediction* traces, which were written before any ground truth was loaded. Information about expected answers appears only in the clearly marked evaluation illustration at the end.

Question (Q12). “What are the leading (by Feb 2026) ML-based interatomic potential methods for large-scale atomistic simulation?”

Step 1: answer family and views. The parser finds the head “methods”, so the admitted types are method, technique, component and cited_work: 2,577 eligible entities. The temporal phrase “by Feb 2026” resolves to the cutoff year 2026. The two views are those of Figure 4.

Step 2: six lists. Each list is computed independently of the others. Table 10 shows the top of each.

Table 10: Top three entries of each first-stage list for Q12, with raw scores, from the frozen channel trace. The lists disagree almost completely, and the raw scores are on unrelated scales.

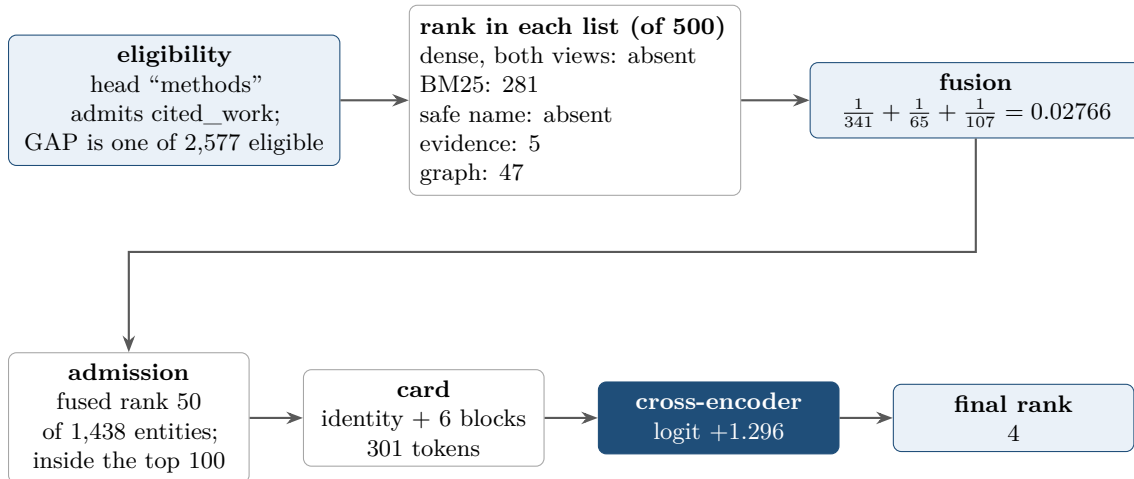
List	Size	Top three (raw score)
Dense, original	500	MatterSim (0.835); Oriented FAST and Rotated BRIEF (0.833); FIRE optimizer (0.830)
Dense, structured	500	MatterSim (0.807); Oriented FAST and Rotated BRIEF (0.801); FIRE optimizer (0.795)
BM25	500	Large-scale atomistic simulation (18.61); Short-time MD simulation workflow (15.69); Type embedding (14.71)
Safe name	1	Large-scale atomistic simulation (32)
Evidence owner	500	Gaussian Approximation Potential (GAP) for silicon (0.856); Gaussian Approximation Potentials (0.852); Orb-v3 (0.852)
Bounded graph	485	the same three entities as the evidence list (0.856; 0.852; 0.852)

Three things are visible in the table. The dense lists place an unrelated entity second, because its context document resembles the question. BM25 and the name list are captured by an entity whose *name* is the question’s task phrase. The evidence and graph lists are the ones that surface interatomic potentials.

Step 3: fusion. The union has 1,438 entities. The fused top three are Gaussian Approximation Potentials (0.0675), MatterSim (0.0628) and Large-scale atomistic simulation (0.0580). The first of these is in no list’s top spot but is ranked 13, 14, 64, 2 and 2 across five lists:

$$\frac{1}{73} + \frac{1}{74} + \frac{1}{124} + \frac{1}{62} + \frac{1}{62} = 0.0675.$$

Step 4: one candidate in detail. Consider the entity **GAP**, a single `cited_work` node. Figure 14 traces it.

**Figure 14:** One candidate’s path through A6 for question Q12, with every number taken from the frozen prediction traces. The entity GAP is invisible to dense retrieval (a three-letter name with little context) and weak in BM25. The evidence channel ranks it fifth because a claim that literally mentions GAP matches the question, and the graph channel reaches it through citing articles. Fusion admits it at rank 50; the cross-encoder, reading the card of Figure 13, moves it to rank 4.

Step 5: the reranked head. The table shows the reranker working and failing in the same list.

Table 11: Final top ten for Q12 from the frozen predictions, with the fused rank each entity had before reranking, its relevance logit and its packed length. Logits order candidates within this question only. For comparison, the two lowest logits in the pool are -7.76 and -10.08 .

Final	Entity	Types	Fused	Logit	Tokens
1	Large-scale atomistic simulation	technique	3	+2.840	63
2	Deep Molecular Dynamics	method	32	+1.766	214
3	ML-IAP	method	49	+1.523	205
4	GAP	cited_work	50	+1.296	301
5	DFT	cited_work	80	+1.265	238
6	unconstrained MLIP evaluation framework	method	52	+1.263	483
7	Gaussian Approximation Potentials	cited_work, method	1	+1.010	237
8	DeepMD	cited_work	62	+0.794	313
9	Machine Learning Interatomic Potentials	method	11	+0.692	117
10	MatterSim	cited_work, method	2	+0.622	256

- **It removes a spurious candidate.** “Oriented FAST and Rotated BRIEF”, second in both dense lists and seventh after fusion, receives a logit of -4.04 and falls to rank 77. Its card holds an identity and one article title.
- **It promotes evidenced candidates.** GAP rises from 50 to 4 and DeepMD from 62 to 8, both on the strength of claims that mention them.
- **It rewards echo.** The top result is a *technique* entity whose name repeats the question’s task phrase. Its card has 63 tokens. A relevance model trained on web search rates a passage that restates the query as highly relevant; it has no notion that the question asks for a potential and not for a task.
- **It ranks near-duplicates separately.** “GAP” and “Gaussian Approximation Potentials”, and “DeepMD” and “Deep Molecular Dynamics”, are distinct canonical entities (Figure 3) and occupy four of the top eight places.

Evaluation illustration (computed after the freeze)

The evaluator, run afterwards in a separate process, maps 13 expected methods for Q12 to graph identities. All 13 are in the 100-candidate pool. Three are in the final top ten: GAP (4), DeepMD (8, the entity named DeepMD) and MatterSim (10); their fused ranks were 50, 62 and 2. The entities at ranks 2 and 7 appear to denote the same methods as two of those targets but have different surface forms, so they receive no credit. No expert adjudicated whether any non-credited candidate is a wrong answer.

14 Temporal and Hypergraph Integrity

V2 retrieves from a temporal hypergraph, so it must respect time and it must be clear about what it does with hyperedges.

14.1 Visibility

V2 uses the main report’s strict visibility rule without modification.

Definition 14.1 (Visibility, inherited). *A node v is visible at year t iff $\text{first_seen_year}(v) \leq t$. A hyperedge is visible at t iff its year is at most t , the publication year of the article asserting it (when recorded) is at most t , and every one of its members is visible.*

`first_seen_year` is the publication year of the earliest corpus paper mentioning the node; it is used instead of `origin_year` so that the snapshot contains only what the *corpus* had seen. Everything downstream inherits the filter: entities are built from visible nodes, evidence links and arcs from visible hyperedges, and an entity’s year is the earliest of its members’. Graph aliases carry no dates, so an alias is used only if its node was last seen no later than the cutoff. **[GUARANTEED]** (tested by `test_temporal_visibility_applies_to_nodes_edges_aliases`)

The cutoff comes from the question. A phrase such as “by Feb 2026” is recognised by a date pattern and reduced to its year; “before Y ” becomes $Y - 1$; the cutoff is the smaller of that year and the configured snapshot (2026). All 18 benchmark questions resolve to 2026, so the frozen run uses a single index. **[MEASURED]**

Year-level temporal precision

The data carry years, not dates. “By February 2026” is therefore enforced as “by the end of 2026”: the system *cannot* exclude material that appeared between March and December 2026. The pretrained models are not historically versioned either, so their parameters may encode knowledge from any time before their own training. This limitation is inherited from the main report and is not solved here.

14.2 Provenance

Every evidence link and every graph path stores the hyperedge id, the relation type, the edge year and the edge’s provenance record (source and asserting article). Every evidence block placed on a card stores its node id, its first-seen year and its provenance. A ranked answer can therefore be traced back to the hyperedges and papers that produced the text the model read.

14.3 What V2 does and does not do with hyperedges

It is important not to blur two different uses of the hypergraph.

- **The hierarchy’s native hyperedge coarsening (P4, T4)** is a rule for what a hyperedge becomes when its endpoints are merged into super-nodes, and the hierarchy’s objective uses that rule. It lives in the main system. **V2 does not redefine, re-implement or evaluate it.**
- **V2’s evidence links and graph expansion** read native hyperedges as they are. A hyperedge of arity n is interpreted through its role pattern (one method and its typed targets; one article, one claim and at most one method), not through a pairwise projection and not through arbitrary co-membership. This supplies structured, provenance-bearing evidence to retrieval. It is a reading of the graph, not a coarsening of it.

The cross-encoder itself is not hypergraph-native in any sense. It is a text model. The graph’s contribution is upstream of it: deciding which entities exist, which evidence belongs to which entity, and which entities are structurally near question-relevant text.

15 Leakage Prevention

Evaluation validity carries the largest weight in the assessment. For a retrieval benchmark the central threat is leakage: any route by which the expected answers could influence the predictions. V2 treats the question text as legitimate input and the answers as evaluation-only, and enforces the distinction mechanically.

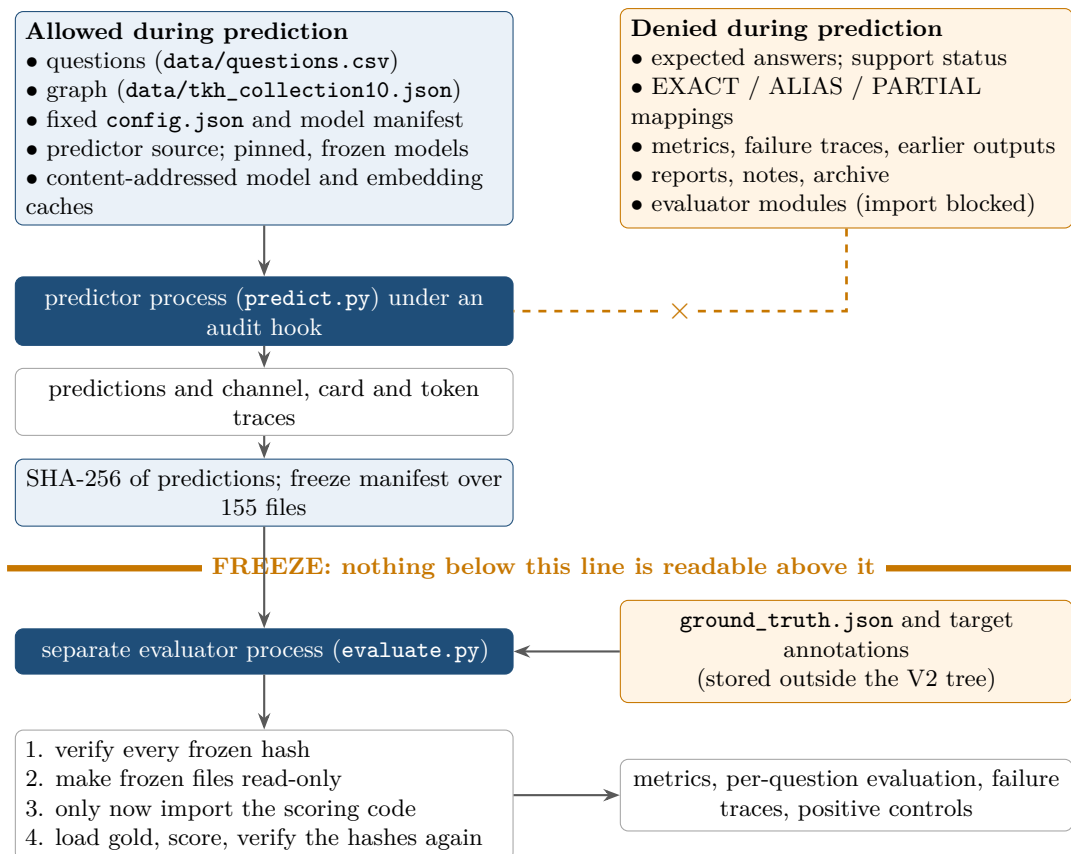


Figure 15: The prediction/evaluation firewall. Above the orange line a single process may read only the inputs listed on the left; any attempt to open a file in the right-hand list, or to import an evaluator module, raises an error. The process ends by hashing its output. Below the line a different process first checks those hashes, protects the frozen files against modification, and only then imports the code that reads the ground truth. Information flows downward only.

15.1 What the prediction phase may and may not see

The predictor may read the question text, the TKH, the pretrained model weights and the fixed configuration. It may *not* read expected answers, target support status, the EXACT/ALIAS/PARTIAL mappings, final metrics, failure traces, or any earlier evaluated output.

This is enforced by a Python audit hook installed before any project code is imported. **[GUARANTEED]** (for file access through Python)

- **Read allowlist.** Inside the V2 tree a read is permitted only for `config.json`, `model_manifest.json`, the two files in `data/`, the predictor’s own source, files the process itself created, and the model and embedding caches. Everything else is refused.
- **Forbidden categories.** Any path containing `ground_truth`, `target_annotations`, `metrics`, `failure`, `oracle`, `positive_controls` and similar markers, and the directories `report/`, `notes/`, `archive/` and `artifacts/evaluation/`, are refused even if a path were otherwise allowed.
- **Outside the tree.** The ground truth lives in the main package, not in V2. Any read of a repository file outside V2, other than installed third-party packages, is refused.
- **Import guard.** Importing a module whose name marks it as evaluation code raises an error before the freeze.
- **Fail closed.** The run asserts at the end that no access was denied; one denied access

aborts the freeze.

Measured audit. The frozen run logged 4,186 project file reads, 0 denied reads, 0 gold reads and 0 reads of earlier evaluation results, and imported 15 local modules, none of them evaluation code. [MEASURED]

15.2 Frozen models and no fitting

- **No fine-tuning.** Both models are used with their published weights; their files are hash-checked against the manifest before every run. [GUARANTEED]
- **No training on benchmark answers.** No component has parameters learned from this benchmark.
- **No learned fusion weights.** Fusion is the unweighted sum of Equation 9.
- **No thresholds fitted to ground-truth rankings.** The constants $k = 60$, $k_1 = 1.5$, $b = 0.75$, the list depth, the seed depths, the graph caps and the card caps are fixed in `config.json`, and the configuration file is part of the hashed freeze. [DESIGN CHOICE] The one exception is the reranking depth: $K = 100$ was chosen among predeclared depths after development results were available (Figure 9).

15.3 Freeze, then score

When the last question is processed, the predictor writes the predictions, computes their SHA-256 and writes a manifest containing the hash of 155 files: the configuration, the model manifest, the input data, the predictor source and every prediction and trace file. It records a `predictions_frozen` event and exits.

The evaluator is a different program run as a different process. Its import order is the safeguard: it imports only hashing utilities, verifies the predictions hash and all 155 file hashes, installs a guard that refuses any write, delete, rename or permission change on a frozen file, and only then imports the scoring module that knows where the ground truth is. After scoring it verifies the hashes once more. [GUARANTEED] (tested by `test_evaluation_freeze_refuses_tampering` and `test_new_prediction_integrity_and_phase_order`)

Freeze first, score second

Predictions are finalised and hashed before evaluation code can read gold.



Figure 16: The order of events recorded in the run’s own event log (UTC, 2 October 2026; spacing not to scale). The ground truth is first opened 5.85 seconds *after* the predictions were hashed, and the hash recorded when evaluation completes equals the hash recorded at the freeze. The same log shows that only one gold-loading event occurred.

Measured integrity. The predictions hash is

```
2d7485f1d0ecfb541d1d83413e2ac1d9
61519ce9c1e0870dbd74311462918570
```

at the freeze, at evaluation start, at evaluation end, and for a second full execution of the predictor into a separate directory. [MEASURED] That second execution reused cached model outputs, so it demonstrates deterministic assembly, not an independent repetition of neural inference.

15.4 What the firewall does not establish

Limits of the isolation

- **Design-time exposure.** The benchmark was evaluated in earlier development iterations, and those results informed human design decisions. The firewall blocks access *while a prediction runs*; it cannot make the benchmark unseen.
- **Configuration chosen with results visible.** Among the predeclared reranking depths evaluated during development (100, 200 and 500), $K = 100$ gave the strongest observed top-10 recovery. This supplement describes that simplified configuration. This selection was made after development results were available and is therefore a development choice, not an independently validated optimum.
- **Cooperative, not hostile-proof.** A Python audit hook observes file access made through Python. It is not an operating-system sandbox and would not contain deliberately adversarial native code.
- **Interrupted attempts.** The event log records three prediction starts; only the last completed and froze. No evaluation ran between them.

16 Evaluation Protocol

16.1 Targets and their status

The benchmark has 18 questions. The 14 Type-A questions list expected methods, giving 63 target occurrences. Identity metrics are computed on these 14; predictions are produced for all 18. Before a target can be scored it must be mapped to the graph, and that mapping is itself uncertain. Each occurrence has one status, assigned by the evaluator after the freeze using the main package’s audited annotations.

Table 12: Target-status categories. These are evaluation concepts: no status, and no mapping, is available to the predictor. [MEASURED] Counts from the frozen `metrics.json`.

Status	Count	Meaning	Direct credit
EXACT	48	A node’s surface form equals the target after normalisation, whatever its type	yes
ALIAS	1	The same identity appears in the graph under a different name (annotated)	yes
PARTIAL	11	Only a related or narrower node exists, e.g. “E(3)-equivariant GNNs”	no
COMPOSITE	1	The target is a combination whose parts are separate nodes	no
ABSENT	2	No defensible mapping was found, e.g. “MACE-F”	no
Total	63	of which directly mapped (EXACT + ALIAS): 49	

Three rules follow, and each lowers the attainable score. [GUARANTEED]

- **PARTIAL is not promoted to success.** Retrieving a related node is not retrieving the expected identity.
- **ABSENT cannot receive identity credit.** There is nothing in the graph to retrieve.

- **COMPOSITE is reported separately.** A composite target counts only if every component is retrieved, and it is never mixed into the direct counts.

Gold incompleteness and the recall ceiling

Because 14 of the 63 occurrences are not directly mapped, final recall can never exceed $49/63 = 0.778$, even for a perfect retriever. Two questions (Q5 and Q11) have no directly mapped target at all and contribute zero by construction. The mapping was produced by automatic matching and assistant inspection; no domain expert adjudicated it. ABSENT means “no mapping found”, not “proven absent”.

16.2 Metrics

Candidate recall@ K (Definition 3.2)

Mapped targets with at least one mapped entity in the fused top K , before reranking, divided by 49.

Final recall $R@k$ (Definition 3.3)

Mapped targets ranked within k , divided by all 63 occurrences, for $k \in \{1, 5, 10, 25, 50, 100\}$. A target mapped to several entities takes its best rank.

Mapped-only recall

The same numerator divided by 49; reported as a secondary view that removes the mapping ceiling.

Mean reciprocal rank

$$\text{MRR} = \frac{1}{14} \sum_{q \in \text{Type A}} \frac{1}{\min_{\tau \in \mathcal{T}_{\text{dir}}(q)} \text{rank}_{\pi_q}(\tau)}, \quad (12)$$

with a term of zero when a question has no mapped target or none is ranked. The denominator is always 14.

Observed and missing ranks

The median and mean rank of mapped targets that receive a rank, and the number that receive none because they never enter the union.

16.3 Is the evaluator itself sound?

A low score could come from a scoring bug. The evaluator therefore runs a positive control, kept separate from all retrieval results: for each question it builds a ranking containing exactly the mapped identities and checks that every one is counted. All 49 are recovered. **[MEASURED]** The control is labelled as gold-informed and evaluation-only and is never reported as a retrieval result.

17 Results

Development diagnostics, not held-out performance

Every number in this section comes from the development benchmark, which was inspected during the design of both V1 and V2. The run is a single deterministic execution on 14 scored questions. The numbers describe this system on this benchmark; they are not estimates of performance on unseen questions, and they carry no confidence intervals.

Table 13: Development diagnostics. A6 with a reranking pool of $K = 100$, frozen ranking key A6_100. Recall counts directly mapped targets ranked within k ; the main column divides by all 63 target occurrences, the secondary column by the 49 that are mapped. Source: frozen `artifacts/evaluation/metrics.json`.

Metric	Hits	Recall (of 63)	Mapped-only (of 49)
R@1	1	0.0159	0.0204
R@5	6	0.0952	0.1224
R@10	12	0.1905	0.2449
R@25	18	0.2857	0.3673
R@50	25	0.3968	0.5102
R@100	37	0.5873	0.7551
MRR (14 questions)		0.1669	
Candidate recall@100 (pool)	37		0.7551
Candidate recall, full union	46		0.9388
Mapped targets with a final rank	46	median rank 40, mean 77.7	
Mapped targets never ranked	3	all removed by the answer-family filter	

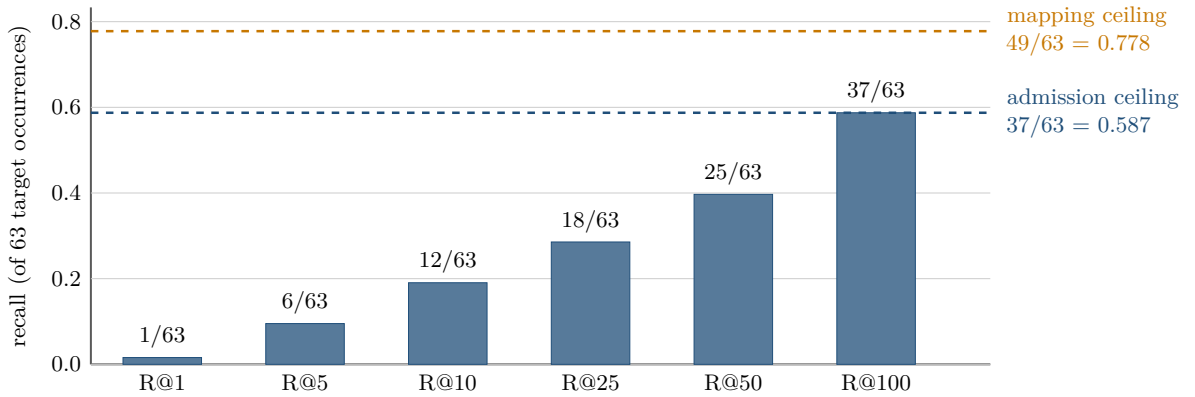


Figure 17: Development diagnostics. Final recall of A6 ($K = 100$) at six cut-offs, as a fraction of all 63 target occurrences, with the hit count above each bar. Two ceilings bound the bars. No ranking could exceed the orange line, because 14 targets have no direct graph identity. This system cannot exceed the blue line at any $k \leq 100$, because only 37 mapped targets are admitted to the pool; R@100 meets that line exactly, as [Proposition 3.1](#) requires.

17.1 Headline numbers

17.2 Where targets are lost

[Figure 18](#) follows the 63 target occurrences through the system. Each step removes targets for a different reason, and the reasons call for different remedies: unmapped targets are a property of the benchmark and the graph, the type filter is a design choice, fusion depth belongs to the admission stage, and the last step is the ordering stage.

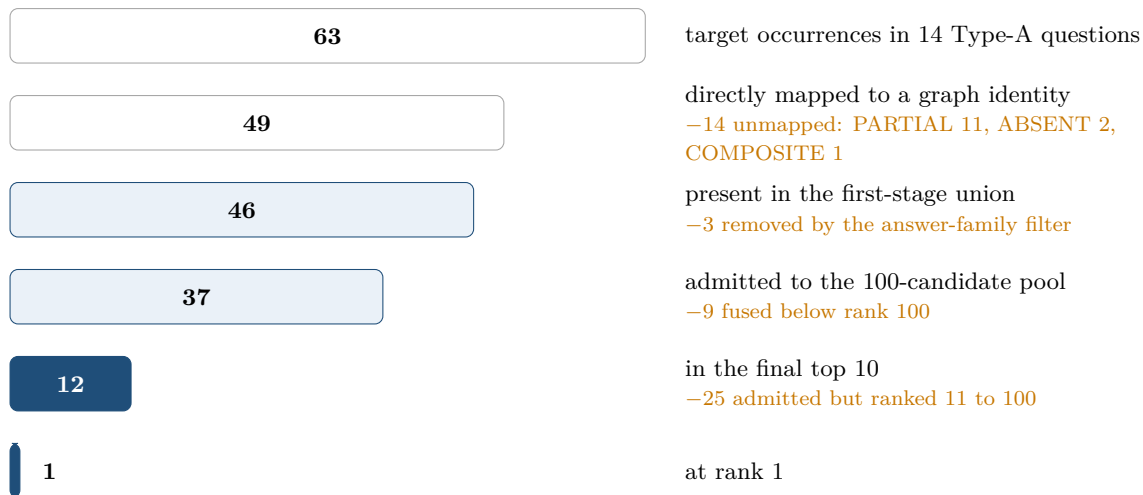


Figure 18: Development diagnostics. The loss of targets stage by stage; bar length is proportional to the count. Fourteen targets are unreachable for any system because the graph holds no direct identity for them. Three are removed by the hard answer-family filter. Nine are in the union but fused below rank 100. Of the 37 admitted, 25 are not brought into the top ten by the cross-encoder. The largest single loss is in ordering, not in admission.

17.3 What the reranking stage changed

Because A6 is its own first stage plus one reranking step, comparing the order before and after that step isolates what the cross-encoder contributes. Both rows of Table 14 are states of the same pipeline on the same pool.

Table 14: Development diagnostics. Mapped-target hits before and after the reranking step of A6, out of 63. R@100 is identical by Proposition 3.1.

Order	@1	@5	@10	@25	@50	@100	MRR
Fused order (input to the cross-encoder)	0	4	6	11	29	37	0.1554
Final order (after the cross-encoder)	1	6	12	18	25	37	0.1669

Among the 37 admitted targets, 17 moved up and 20 moved down; none stayed in place. Seven entered the top ten and one left it. The median rank of admitted targets improved from 36 to 27. **[MEASURED]** The reranker improves the head of the list (hits at 10 double, from 6 to 12) but it is not uniformly beneficial: hits at 50 fall from 29 to 25, because some admitted targets are pushed toward the bottom of the pool. The largest such move is MACE for Q12, from fused rank 42 to final rank 86. **[MEASURED]**

17.4 Per-question view

The aggregate hides strong concentration. Q12 and Q14 hold 24 of the 49 mapped targets and 7 of the 12 top-ten hits. Six of the 14 questions have a mapped target in the top ten; ten have one in the top 100. Q13 is the clearest admission failure: all eight of its targets reach the union, but six are fused below rank 100. **[MEASURED]**

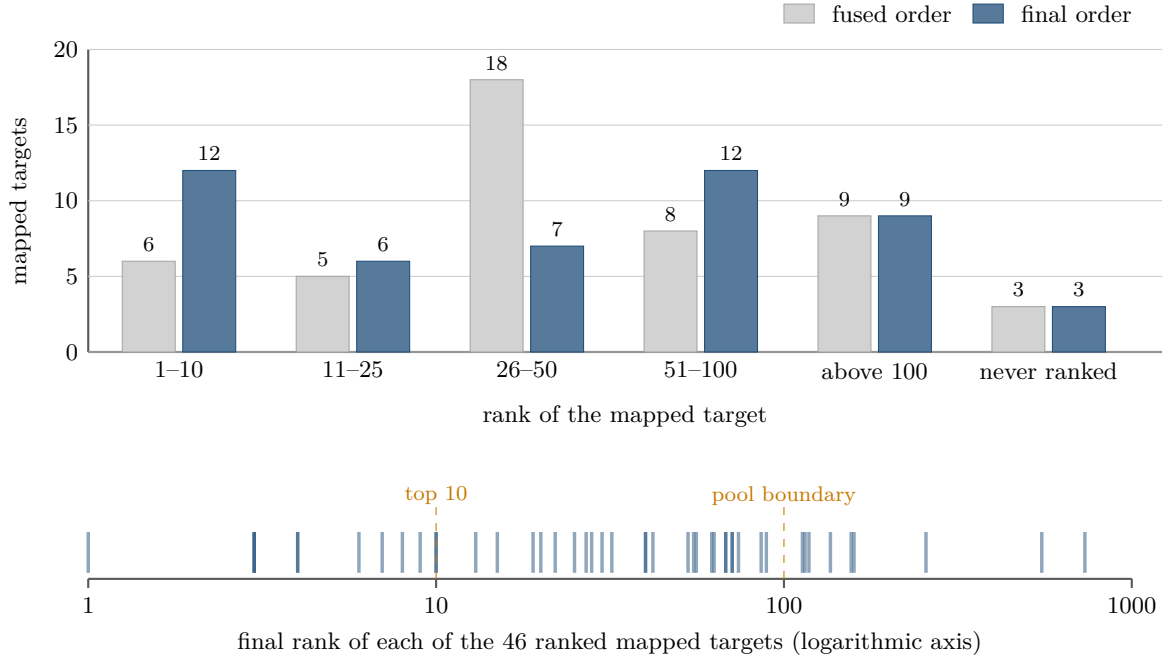


Figure 19: Development diagnostics. Rank distribution of the 49 directly mapped targets. Top: counts per rank band before reranking (grey) and after it (blue). The cross-encoder moves mass out of the 26–50 band in both directions: into the top ten and into the 51–100 band. The last two bands are identical by construction, since reranking touches only the first 100 positions. Bottom: each ranked target’s final position as one tick on a logarithmic axis; darker ticks are coincident ranks. Nine ticks lie beyond the pool boundary and were never seen by the cross-encoder.

Table 15: Development diagnostics per Type-A question for A6 ($K = 100$). “Mapped” is the number of targets with a direct graph identity; “union” and “pool” count how many of those survive each admission step; “first” is the best final rank of a mapped target.

Question	Targets	Mapped	In union	In pool	Top 10	First
Q1	7	7	7	6	1	3
Q2	3	2	2	2	2	7
Q3	2	1	1	1	1	3
Q4	3	2	2	2	0	40
Q5	2	0	0	0	0	–
Q6	3	1	1	1	0	22
Q7	2	1	1	1	0	71
Q8	2	1	1	1	1	6
Q9	3	1	1	0	0	256
Q10	2	1	1	0	0	156
Q11	2	0	0	0	0	–
Q12	13	13	13	13	3	4
Q13	8	8	8	2	0	62
Q14	11	11	8	8	4	1
Total	63	49	46	37	12	

18 Interpreting the Results

18.1 What A6 demonstrates

- **Entity-centric, multi-channel retrieval admits answer identities that entity-level dense retrieval alone does not.** Within the same entity universe, the dense lists cover 19 and 21 of 49 mapped targets, the evidence-owner list 44, and the union 46 (Table 8).
[DEVELOPMENT OBSERVATION]
- **Evidence-to-entity retrieval is the mechanism that addresses the original failure.** The texts that used to outrank the answer now carry their similarity to it. The worked example shows this for an entity that dense retrieval cannot see at all.
- **A cross-encoder allows deeper query–candidate interaction and improves the head of the list on this benchmark:** hits at 10 rise from 6 to 12 on the same pool.
[DEVELOPMENT OBSERVATION]
- **The pipeline is auditable.** Every final rank can be traced to channel ranks, a fusion sum, a card with logged inclusions and omissions, a token count and a logit.

18.2 What A6 does not demonstrate

- **It does not solve the task.** The system still misses most expected answers at useful depths: 51 of 63 are outside the top ten and the first result is a mapped target for one question out of 14.
- **Exact answer ranking remains difficult.** Most of the loss is in ordering (Figure 18), and the reranker also demotes admitted targets (Table 14).
- **No held-out claim.** Benchmark exposure makes these development results. The pool depth in particular was fixed with them in view.
- **No statistical claim.** Fourteen questions, one run, no intervals. A difference of a few hits is within what a handful of questions can produce.
- **No claim about the hierarchy.** V2 does not use it.
- **No comparison with V1 on equal terms.** V1 ranked 5,480 node documents and V2 ranks at most 2,577 eligible canonical entities with a different scoring rule. The two were not run as a matched experiment, so their numbers should not be subtracted from each other.

Agreement with gold is not correctness, and disagreement is not error

A high-ranked candidate that is not an annotated target has not been shown to be wrong. The expected-method lists are short and were not written as exhaustive; several high-ranked non-credited entities in Table 11 are plausible answers or duplicates of credited ones. Equally, a credited target at rank 4 does not show that the system “understood” the question. Only an expert judgement of the returned lists could settle either point, and none was made.

18.3 Verified versus assumed

Table 16 sorts the statements of this report by the kind of support they have.

Table 16: Separation of what was verified from what was assumed or chosen.

Verified (read from frozen artifacts, or enforced and tested in code)	Assumed, chosen or not established
Candidate counts: 46/49 in the union, 37/49 in the pool	That the answer-family map reflects what question authors mean
Top- k recovery: 1, 6, 12, 18, 25, 37 of 63	That $K = 100$ is a good operating point beyond this benchmark
Prediction hash equal at freeze, evaluation and replay	That the graph path restrictions keep the right paths
Graph and index counts (Table 5)	That the evidence caps show the most useful evidence
Pinned model revisions and file hashes	That a web-search relevance model is a useful prior for materials-science method selection
0 denied reads, 0 gold reads during prediction	That node types and years in the source data are correct
57 tests passed	That the target mapping is complete and correct (no expert adjudication)
Reranking permutes only the first 100 positions	That non-credited high-ranked candidates are wrong (not established)
Evaluator recovers 49/49 forced identities	Held-out generalisation; month-level temporal validity

19 Efficiency

The two-stage design is motivated by cost, so the cost should be stated. The run is a single CPU process with four threads. Table 17 lists the logical work per question, and Figure 20 shows where it goes.

Table 17: Logical work per question, and timing where the frozen run recorded it. Counts are comparisons, not latency. [MEASURED] Sources: `audit/efficiency.json` and `evaluation/efficiency_report.json`.

Operation	Per question	Note
Dense inner products	10,960 (5,480 documents $\times$ 2 views)	The run also scored a third, diagnostic view not used in fusion; logged total 16,440
Evidence inner products	8,548 (4,274 items $\times$ 2 views)	Reused by the evidence channel, the graph channel and card selection
BM25	2,978 lexical documents	1,637,900 document-term lookups over 18 questions
Name checks	one per eligible entity	at most 2,978
Graph expansion	200 seeds, fan-out ≤ 64 , ≤ 2 hops	bounded by construction
Cross-encoder pairs	100	1,800 over 18 questions
Cross-encoder time, 100 pairs	7.7 s mean (3.8 to 11.4 s)	Sum of logged amortised batch times for the first 100 fused candidates; an estimate
First-stage time	0.83 s mean (0.14 to 1.13 s)	With corpus vectors already cached; includes trace writing

Three cautions apply.

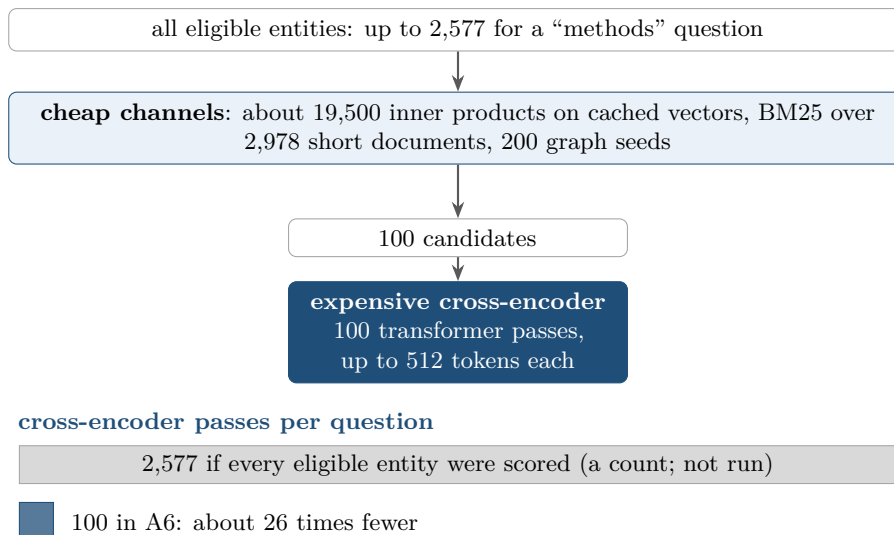


Figure 20: Where the work goes. The first stage touches every eligible entity but only with operations on precomputed vectors and term statistics. The cross-encoder is confined to 100 candidates. The bars compare the number of joint forward passes per question with and without the first stage; the longer bar is a count, not an experiment that was run.

- **Comparisons are not production latency.** They count operations; they say nothing about batching, hardware or serving overhead.
- **CPU timing is machine-specific.** The times are from one CPU run on one machine and mix cached and uncached work. The 100-pair time is assembled from amortised batch timings of a run that scored a deeper prefix; it was not measured as a stand-alone 100-pair run.
- **Nothing here is a scalability claim.** The corpus is 52 papers. Exhaustive inner products and a pure-Python BM25 are adequate at this size and would need replacing at a larger one.

20 Reproducibility

Table 18: Pinned models and environment of the frozen run. **[MEASURED]** Sources: `config.json`, `model_manifest.json` and `audit/model_version_manifest.json`.

Item	Value
Bi-encoder	BAAI/bge-small-en-v1.5
revision	5c38ec7c405ec4b44b94cc5a9bb96e735b38267a
Cross-encoder	cross-encoder/ms-marco-MiniLM-L6-v2
revision	233902d25c440f23af6f7d6e94d2946bac0bee0a
Model files	every file hashed (SHA-256) in the manifest and checked before prediction
Python	3.11.2 (Windows, 64-bit)
Libraries	numpy 2.3.5; torch 2.4.1; transformers 4.57.6; sentence-transformers 5.1.0; huggingface-hub 0.36.2
Device	CPU, 4 threads; offline mode, local files only
Seed	42 for Python, NumPy and PyTorch; deterministic algorithms enforced
Batch sizes	32 (encoder), 16 (cross-encoder)
Full dependency closure	<code>requirements.txt</code> (exact versions)

Determinism. Every sort has an explicit tie-breaker (entity id, evidence id or seed id). Fusion accumulates channels in sorted name order. BM25 iterates over query terms in sorted order, so its floating-point sums do not depend on Python’s hash seed (tested by `test_bm25_independent_of_python_hash_seed`). No sampling occurs anywhere.

Frozen artifacts. The freeze manifest covers 155 files. A second execution of the predictor into a separate directory reproduced the predictions byte for byte, with the same SHA-256 (Section 15); it reused cached model outputs. **[MEASURED]**

Process separation. Prediction and evaluation are different entry points (`scripts/predict.py`, `scripts/evaluate.py`). The predictor refuses to overwrite an existing freeze; a new run must name a new output directory.

Tests. The V2 suite has 57 tests; all passed, none failed. **[MEASURED]** The suite covers the whole V2 package, including parts outside this document’s scope. Those most relevant to A6 are listed in Table 19.

Table 19: Tests that guard the properties this document relies on (names as in the repository).

Test	Property
<code>test_grouping_deterministic_and_no_fuzzy_alias_merges</code>	Canonical entities
<code>test_temporal_visibility_applies_to_nodes_edges_aliases</code>	Visibility
<code>test_safe_names_match_boundaries_and_punctuation</code>	Name channel
<code>test_graph_bounded_paths_and_hubs</code>	Graph caps and path shapes
<code>test_rrf_is_rank_only_and_order_independent</code>	Fusion
<code>test_query_dependent_selection_and_deduplication</code>	Card selection
<code>test_identity_survives_token_packing</code>	Identity kept in the card
<code>test_all_cross_encoder_identities_and_token_limits</code>	512-token limit
<code>test_reranker_only_changes_frozen_prefix</code>	Head/tail rule
<code>test_prediction_source_has_no_gold_or_evaluator_import</code>	Isolation
<code>test_runtime_guard_blocks_protected_writes_and_gold_reads</code>	Isolation
<code>test_evaluation_freeze_refuses_tampering</code>	Freeze
<code>test_full_cached_replay_is_identical_when_available</code>	Replay

Commands. From the repository root, in PowerShell. These are the README’s commands with the interpreter path abbreviated as `$py`.

```
python -m venv clean_tshc_v2/.venv
$py = "clean_tshc_v2/.venv/Scripts/python.exe"
& $py -m pip install -r clean_tshc_v2/requirements.txt
& $py -B clean_tshc_v2/scripts/prepare_models.py
& $py -B clean_tshc_v2/scripts/predict.py
& $py -B clean_tshc_v2/scripts/evaluate.py
& $py -B clean_tshc_v2/scripts/predict.py --output artifacts/reproduction
& $py -B -m pytest clean_tshc_v2/tests -q -p no:cacheprovider
```

Model preparation uses local files unless run with `-download`; it never substitutes a different revision. The A6 ranking with $K = 100$ is the entry `rankings["A6_100"]` of each question in `artifacts/prediction/retrieval/predictions.json`, and its metrics are under `systems["A6_100"]` in `artifacts/evaluation/metrics.json`.

21 Limitations

The three limitations that matter most

1. **Development benchmark.** The questions and answers were seen during design. Nothing here measures performance on unseen questions.
2. **Incomplete and unadjudicated gold.** Fourteen of 63 targets have no direct graph identity, and a high-ranked non-gold candidate is not known to be wrong.
3. **General-purpose relevance model on thin evidence.** The reranker was trained on web search, and most entities have almost no evidence to show it.

The complete list:

1. **Development benchmark, not held out.** Earlier iterations were evaluated on the same questions; run-time isolation does not undo that.
2. **Small number of scored questions.** Fourteen Type-A questions, 63 target occurrences, one deterministic run, no confidence intervals. Two questions hold about half of the mapped targets (24 of 49).
3. **Some gold entities are PARTIAL, ABSENT or COMPOSITE.** They can never earn direct credit, capping recall at 49/63.
4. **Hard answer-type filtering can exclude valid entities** when the graph’s typing disagrees with the question’s wording. Three expected datasets were removed this way.
5. **Entity-specific evidence coverage is incomplete.** 2,508 of 2,978 entities have article titles only. The conservative safe-name rule adds to this: names such as “CHGNet” are not recognised as identifiers and gain no mention evidence.
6. **The cross-encoder is trained on general relevance data,** not on materials-science method selection. It rewards restatement of the question (Table 11).
7. **Relevance logits are not calibrated probabilities.** They order candidates within one question and mean nothing across questions.
8. **Evidence associated through graph structure does not entail a scientific claim.** Most graph paths are citation paths.
9. **Year-only dates cannot enforce “February 2026”.**
10. **A high-ranked non-gold candidate is not demonstrated to be wrong.**
11. **$K = 100$ is a development configuration.** Among the predeclared reranking depths it gave the strongest observed top-10 recovery, and it was selected after the development results were available. It is not the frozen run’s primary configuration and not an independently validated or universal optimum.
12. **No model fine-tuning was performed,** by design and in line with the assessment’s scope. A domain-adapted reranker might behave differently.
13. **No claim of production latency or scalability** beyond this 52-paper corpus.
14. **Conservative identity grouping fragments answers.** Near-identical entities with different surface forms are ranked and scored separately.
15. **The structured view comes from a heuristic parser** that can mis-split a question; only the parallel verbatim view protects against this.

22 What Four More Weeks Would Add

None of the following has been implemented, and no result is claimed for any of it. In priority order:

1. **Independent expert evaluation of retrieved answers.** Have a domain expert judge the top results per question blind to the gold list. This replaces “agreement with an incomplete list” by an actual precision estimate and settles whether non-credited candidates are errors.
2. **A genuinely held-out query set,** written before looking at any retrieval output, with the configuration (including K) fixed in advance.
3. **Better graph evidence coverage.** Most entities have nothing candidate-specific to show. Linking more claims to the entities they describe, with the same role checks, would give both the evidence channel and the card something to work with.
4. **Scientifically specialised reranking.** Evaluate a relevance model adapted to scientific text, declared before scoring, against the general-purpose one.
5. **Softer or evidence-aware answer-family constraints,** so that an entity typed as a component but evidenced as a dataset is demoted instead of deleted.
6. **Larger-corpus performance profiling,** to learn where exhaustive scoring stops being adequate.
7. **Method-level answer synthesis,** only after entity retrieval is trustworthy. Generating prose answers on top of an unreliable ranking would hide its errors.
8. **A matched comparison with hierarchy routing,** holding entities, evidence, scorers, snapshot, mapping and cost accounting fixed, so that the main report’s open extrinsic question can be answered on equal terms.

23 Conclusion

The main report showed that hierarchy construction can be evaluated rigorously, and its initial downstream retriever exposed an answer-identity bottleneck: dense similarity found the right scientific region and ranked the texts describing a solution above the name of the solution.

V2 responds by separating two tasks. *Candidate admission* uses complementary semantic, lexical, evidence and graph signals over canonical entities to form a high-recall pool, and reciprocal rank fusion combines their rankings without calibrating their scores. *Candidate ordering* applies a joint relevance cross-encoder, the one expensive operation, to only the top 100 candidates, each presented as an identity-first, query-dependent card of provenance-bearing evidence.

On the development benchmark the pool admits 37 of 49 mapped targets and the final ranking places 12 of 63 targets in the top ten. The result is imperfect and development-only: ordering loses more targets than admission, the reranker rewards restatement of the question, evidence is thin, and the gold lists are incomplete. It is nevertheless an auditable, reproducible retrieval architecture, with predictions frozen before scoring, that directly addresses the failure exposed by the original extrinsic evaluation, and it leaves the hierarchy and its guarantees exactly as the main report established them.

AI assistance

This supplement was drafted with an AI writing assistant from the current V2 source and the frozen artifacts listed in [Appendix B](#); every number was read from those artifacts. AI use in the V2 implementation is documented in `AI_USAGE.md` of the V2 package.

References

- P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, M. Rosenberg, X. Song, A. Stoica, S. Tiwary and T. Wang. MS MARCO: A human generated MACHine Reading COMprehension dataset. *arXiv:1611.09268*, 2016.
- L. Burduli. Temporal Semantic Hypergraph Coarsening: Multi-resolution semantic abstraction over an evolving knowledge hypergraph. Technical research report, September 2026. Package `clean_tshc`, file `report/full_report.pdf`.
- G. V. Cormack, C. L. A. Clarke and S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proc. 32nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi:10.1145/1571941.1572114.
- J. Devlin, M.-W. Chang, K. Lee and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019. doi:10.18653/v1/N19-1423.
- R. Nogueira and K. Cho. Passage re-ranking with BERT. *arXiv:1901.04085*, 2019.
- N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. EMNLP-IJCNLP*, pages 3982–3992, 2019.
- S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi:10.1561/15000000019.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.
- W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33*, 2020.
- S. Xiao, Z. Liu, P. Zhang and N. Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv:2309.07597*, 2023.

A Configuration Constants

Table 20: Constants used by A6, from the frozen `config.json`. All are fixed settings and none was fitted, except that the reranking depth K was selected among predeclared depths after development results were available (Figure 9). [DESIGN CHOICE]

Field	Value	Role
<code>snapshot</code>	2026	Latest visibility cutoff
<code>seed, threads</code>	42, 4	Determinism and CPU threads
<code>context_cap</code>	12	Neighbours in a dense node document
<code>bm25_k1, bm25_b</code>	1.5, 0.75	BM25 saturation and length normalisation
<code>channel_depth</code>	500	Truncation of each first-stage list
<code>evidence_seed_depth</code>	500	Evidence seeds for the evidence channel
<code>graph_seed_depth</code>	200	Evidence seeds for graph expansion
<code>graph_max_hops</code>	2	Maximum path length
<code>graph_fanout_cap</code>	64	Arcs followed from one node
<code>graph_article_degree_cap</code>	80	Largest article usable as an intermediate
<code>graph_path_decay</code>	0.8	Factor per hop beyond the first
<code>rrf_k</code>	60	Fusion constant
<code>cross_encoder_max_tokens</code>	512	Input limit of the reranker
<code>query_evidence_caps</code>	see Table 9	Blocks per evidence family
<code>answer_families</code>	see Table 6	Answer head to node types
Reranking pool K	100	Ranking key A6_100. The field <code>rerank_k = 500</code> is the frozen run’s primary depth; 100 is one of its declared <code>rerank_controls</code> , selected after development results were available

B Artifacts and Sources Inspected

Every numerical or implementation statement in this document was taken from the files below, in this order of precedence: frozen prediction and evaluation artifacts; current V2 source and configuration; current V2 README and reports; the main report for inherited hierarchy properties. No experiment was run and no file other than this document was written.

Table 21: Sources, relative to `clean_tshc_v2/` unless stated.

File	Used for
<code>artifacts/evaluation/metrics.json</code>	Recall, MRR, candidate recall, status counts, channel coverage, reranking movement
<code>artifacts/evaluation/per_question_evaluation.json</code>	Per-question and per-target ranks
<code>artifacts/evaluation/efficiency_report.json</code>	Logical work totals
<code>artifacts/evaluation/audit/evaluation_manifest.json</code>	Hashes at evaluation
<code>artifacts/prediction/retrieval/predictions.json</code>	Fused and final rankings
<code>artifacts/prediction/retrieval/channels/</code> <code>artifacts/prediction/retrieval/selected_evidence/</code>	Per-channel ranks, scores, graph paths Selected and omitted card evidence
<code>artifacts/prediction/retrieval/cross_encoder/</code>	Packed blocks, token counts, logits
<code>artifacts/prediction/query_understanding/</code>	Query views, answer heads, cutoffs
<code>artifacts/prediction/entity_index/2026.json</code>	Entities, evidence links, arcs
<code>artifacts/prediction/audit/</code>	Freeze manifest, event log, access audit, efficiency, model versions
<code>artifacts/reproduction/retrieval/predictions.sha256</code>	Replay hash
<code>artifacts/verification/</code>	Test log, final verification
<code>config.json</code> , <code>model_manifest.json</code> , <code>requirements.txt</code>	Constants, revisions, hashes, versions
<code>src/tkh_abstraction_v2/*.py</code> , <code>scripts/</code> <code>predict.py</code> , <code>scripts/evaluate.py</code>	Implementation behaviour
<code>tests/*.py</code>	Test names
<code>README.md</code> , <code>notes/ASSESSMENT_ALIGNMENT.md</code> , <code>report/</code>	Commands, scope, existing V2 reports
<code>../clean_tshc/report/full_report.tex</code>	Inherited hierarchy properties, V1 retrieval result, style
<code>../clean_tshc/data/ASSESSMENT.md</code>	P1–P6, T1–T7, scoring weights

C Glossary

A6

The configuration described here: fused candidates reranked by a relevance cross-encoder; pool $K = 100$.

Answer family

The node types admitted for a question’s answer head.

Bi-encoder

A model that embeds each text separately; scoring is a vector product.

BM25

A lexical scoring function based on term frequency, inverse document frequency and length normalisation.

Candidate card

The text shown to the cross-encoder for one candidate: identity, then selected evidence.

Candidate pool

The first K entities of the fused order; the only entities the cross-encoder scores.

Canonical entity

An equivalence class of graph nodes with the same normalised name in a compatible type family.

Cross-encoder

A model that reads question and candidate jointly and outputs one score.

Evidence item

A claim, task, problem, technique, component, dataset, metric or article node whose text can support an entity.

Freeze

Hashing predictions and their traces before the evaluator may run.

Relevance logit

The cross-encoder's unbounded output; an ordering signal, not a probability.

RRF

Reciprocal rank fusion: a sum of $1/(k + \text{rank})$ over lists.

Seed

One of the evidence items most similar to the question, from which the evidence and graph channels start.

TKH

Temporal Knowledge Hypergraph, the supplied data.